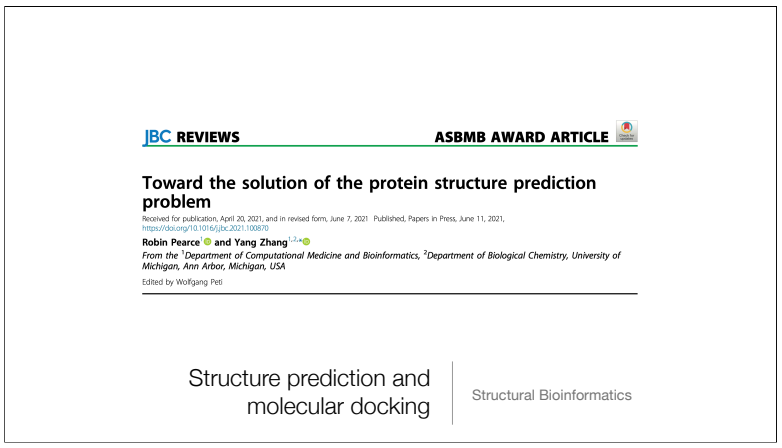
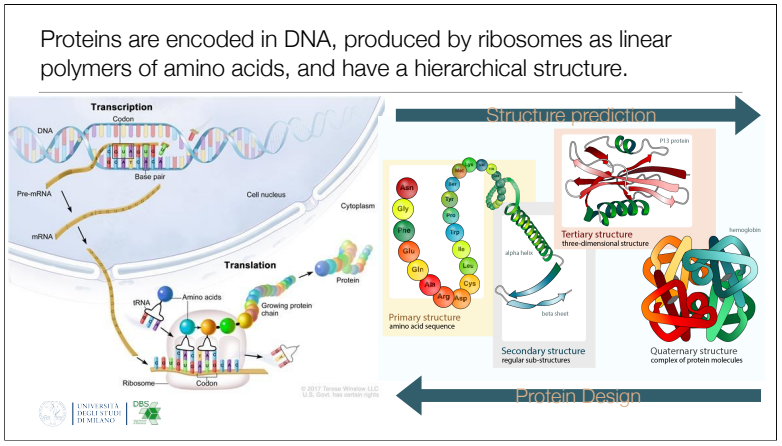



Main idea: this lecture addresses two closely related questions. Given a sequence, what is its 3D structure (structure prediction)? Given two molecules, where and how strongly do they bind (molecular docking)?
Roadmap: (1) why the sequence determines the structure, and why evolution links sequence, structure and function; (2) the data: UniProt vs PDB; (3) classical approaches: secondary structure prediction, physics-based simulations, homology modelling, coevolution; (4) machine learning: from distograms to AlphaFold2 and its successors, and how to read their confidence metrics; (5) molecular recognition and docking: pose search, scoring functions, Monte Carlo, virtual screening.
Link to previous lectures: Lecture 2 ended with the message that every method in the course has to solve two intertwined problems, SAMPLING the relevant configurations and defining an ENERGY (score, loss) that tells good from bad. Every method in this lecture is a different answer to these two problems. Lecture 3 (T. Giorgino) gave us the language of neural networks, loss functions, training and overfitting, which we use here without re-deriving it.
The review by Pearce and Zhang (JBC 2021) shown here is one of the three reference papers of the course: read it before the exam.
Practicals: T02 (structure prediction of a complex: SWISS-MODEL, ColabFold, AlphaFold3, Boltz) and T03 (docking with AutoDock Vina) are built on this lecture.

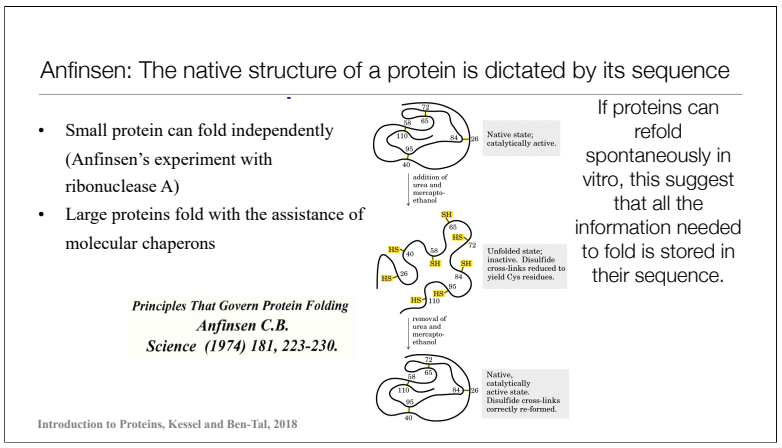



Main idea: protein structure is hierarchical. Structure prediction is the forward problem (sequence → structure), protein design is the inverse problem (structure/function → sequence).

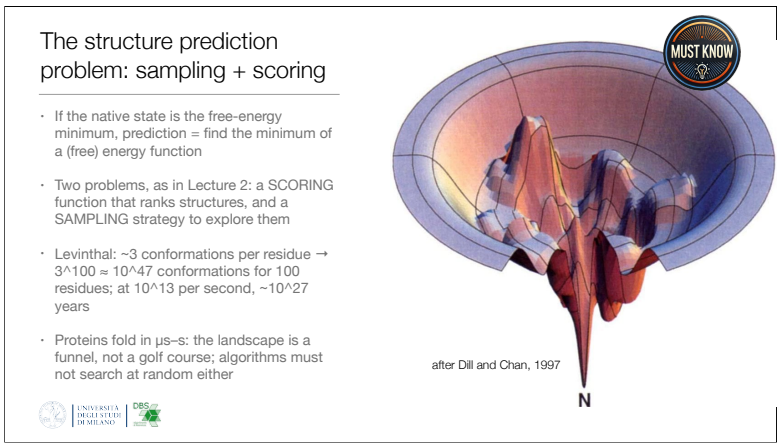
Explanation: DNA is transcribed into mRNA, translated by the ribosome into a linear chain of amino acids (primary structure). The chain forms local regular elements (secondary structure: α-helices, β-strands), packs them in 3D (tertiary structure) and can assemble with other chains (quaternary structure, e.g. haemoglobin, an α2β2 tetramer). Prediction moves left to right on this slide; design (Lecture 9) moves right to left, and the two problems share most of their tools.

To appreciate the difficulty: the input is just a string of letters, the output is a set of 3D coordinates. A 300-residue protein has about 600 backbone dihedral angles (ϕ, ψ) plus the side chains: a structure is a point in a space with hundreds to thousands of dimensions.

Remember: input = sequence (1D), output = coordinates (3D). The question for the next slide: is the sequence really enough?




Main idea: Anfinsen's thermodynamic hypothesis. Under physiological conditions the native structure is the state of lowest free energy, therefore it is determined by the sequence alone.
Explanation: Ribonuclease A (124 residues, 4 disulfide bonds) is unfolded with urea and its disulfides reduced with β-mercaptoethanol: activity is lost. When both are removed slowly, activity is recovered and the 4 correct disulfides re-form, out of 105 possible pairings. If instead the protein is re-oxidised while still in urea, disulfides form at random ("scrambled" RNase, about 1% activity); adding traces of β-mercaptoethanol lets the bonds reshuffle until the native, most stable, pairing is reached. The native state is a thermodynamic minimum, not a kinetic accident.
Link to Lecture 2: the native state is the macrostate with the lowest free energy, hence the most populated. Folding free energies are small (typically 5-15 kcal/mol, i.e. about 10-25 kT), the result of many weak interactions: proteins are only marginally stable.
Caveats: large proteins often need chaperones, but chaperones do not encode the structure, they prevent misfolding and aggregation (a kinetic help), so Anfinsen's principle still holds. And the "native structure" is really an ensemble (Lecture 1); about 30% of residues in eukaryotic proteomes are intrinsically disordered.
Bridge to the next slide: if the native state is a free-energy minimum, structure prediction becomes an optimisation problem.
Remember: the sequence encodes the structure because the native state is the most stable one, under physiological conditions.

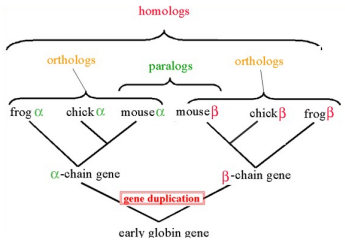



Main idea: if the native state is the free-energy minimum (Anfinsen), structure prediction is an optimisation problem with the two ingredients that, as we saw at the end of Lecture 2, every method in this course has: a SCORING function and a SAMPLING strategy.
Explanation: the scoring function U(structure) should have its minimum at the native structure: a physics-based force field, a knowledge-based statistical potential, or a function learned by a network. The sampling strategy explores conformations to find that minimum. Levinthal's paradox shows why sampling is the hard part: with ~3 conformations per residue, a 100-residue chain has 3^100 ≈ 10^47 conformations; testing 10^13 per second would take ~10^27 years, while real proteins fold in microseconds to seconds.
The figure (energy landscape after Dill and Chan): the width of the funnel is the conformational entropy (how many conformations), the depth is the energy; N is the native state at the bottom. Many routes lead down, so the protein never needs to visit all conformations; the bumps are local minima where folding (and our algorithms) can get trapped. Link to Lecture 2: this is a free-energy landscape, F = E - TS.
Remember: every method in this lecture is a way to make sampling tractable: copying templates (homology modelling), assembling fragments (Rosetta), restraining contacts (coevolution), or learning from the PDB (deep learning).

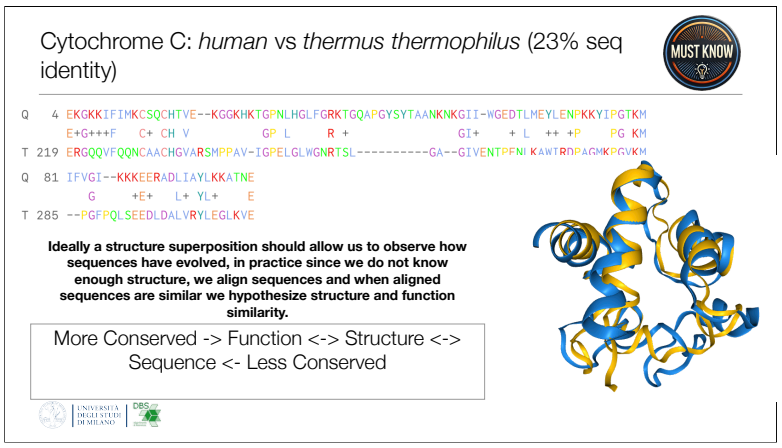
Protein structure and function are intimately linked

Upon natural evolution, through sequence modifications, structural motifs performing specific functions have been reused and adapted to work in different contexts.

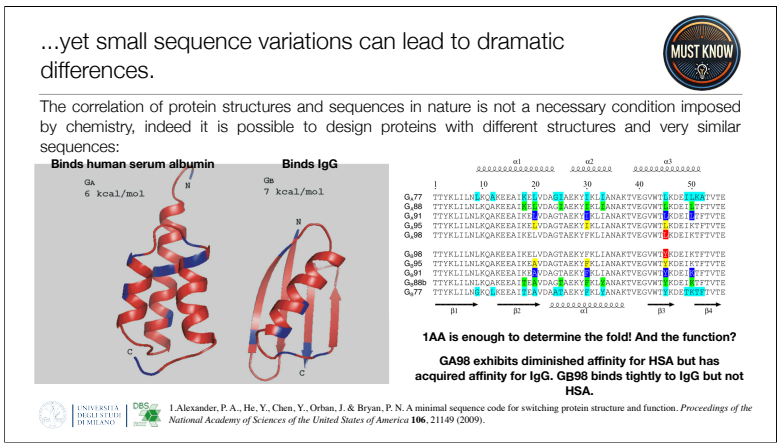
As a result if one takes proteins performing specific functions, even in very unrelated organisms, they still show high structural similarity and some sequence similarity.



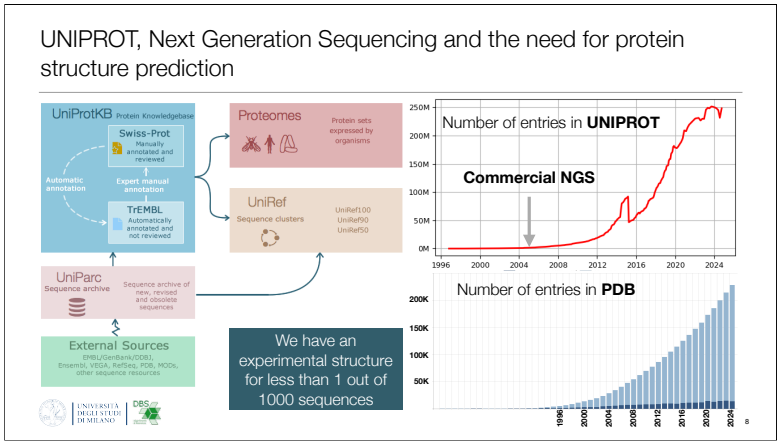

Main idea: evolution reuses structures. Proteins with the same function in different organisms share their structure and some sequence similarity.
Explanation: reading the globin tree: homologs are proteins that share a common ancestor. Orthologs are separated by a speciation event (frog α vs chick α globin) and usually keep the same function. Paralogs are separated by a gene duplication within a genome (α vs β globin) and can diverge in function. Mutations that destroy the fold, and therefore the function, are removed by natural selection: this is why structure is conserved for much longer than sequence.
Why it matters for this lecture: it is the implicit assumption behind homology modelling, multiple sequence alignments (MSA) and coevolution. The ortholog/paralog distinction comes back when we pair sequences to predict complexes ("Coevolution also works for protein complexes", and AlphaFold-Multimer in T02).
Remember: homology is a yes/no statement about common ancestry. "70% homology" is wrong: say 70% sequence identity (or similarity).




Main idea: structure is more conserved than sequence. Human and Thermus thermophilus cytochromes c share only 23% sequence identity but have nearly the same fold.
Explanation: the alignment is a BLAST-like output: letters on the middle line are identities, "+" are similar (conservative) substitutions. Despite billions of years of divergence and low identity, the superposition (blue vs yellow) shows the same fold around the heme. Hierarchy of conservation: function > structure > sequence.
Rules of thumb (Rost 1999): above ~30-40% identity over 100 or more aligned residues two proteins almost certainly share the fold, and homology modelling is reliable; 20-30% is the "twilight zone", where sequence alone can no longer distinguish homologs from random similarity; below 20% (the "midnight zone") structures can still be similar, but sequence comparison cannot detect it. Profile and HMM methods (PSI-BLAST, HHpred) push detection deeper into the twilight zone.
In practice, since we have far fewer structures than sequences, we align sequences and, when they are similar, we transfer structure and function.
Remember: high sequence identity → same structure; low identity does not exclude the same structure. The reverse is the subject of the next slide.

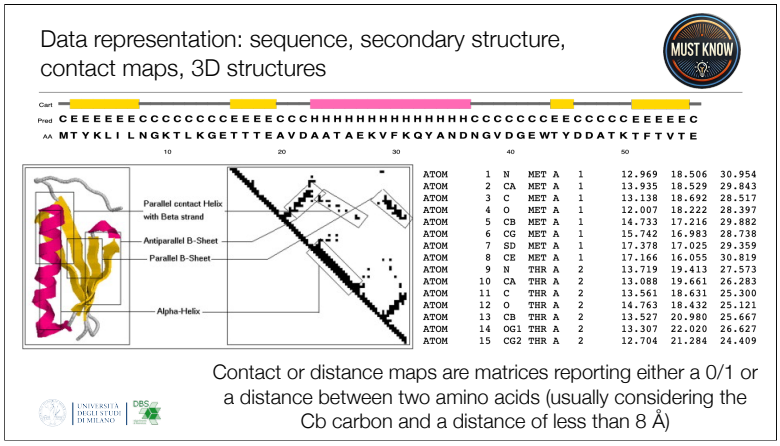



Main idea: the sequence-structure relationship is not imposed by chemistry: two designed proteins with almost identical sequences can have different folds and different functions.
Explanation: Bryan, Orban and co-workers started from two domains of streptococcal protein G: GA (a three-helix bundle that binds human serum albumin) and GB (a 4β+1α fold that binds IgG). By mutating them step by step towards each other they obtained GA98 and GB98, which are 98% identical: they differ by a single residue out of 56 (position 45, Leu in GA98, Tyr in GB98), yet they fold differently and bind different partners. Both have stabilities of a few kcal/mol (6 and 7 kcal/mol in the figure).
Link to Lecture 2: populations depend exponentially on free-energy differences; a single mutation that shifts the balance by a few kcal/mol is enough to switch the dominant macrostate entirely.
In nature such cases are rare (fold-switching proteins such as RfaH, lymphotactin or KaiB) because evolution proceeds through functional intermediates. They are a known failure case for predictors that rely on evolutionary similarity: AlphaFold2 usually predicts only the dominant fold (Chakravarty and Porter, Protein Science 2022).
Remember: "similar sequence → similar structure" is an empirical, evolutionary rule, not a law of chemistry.

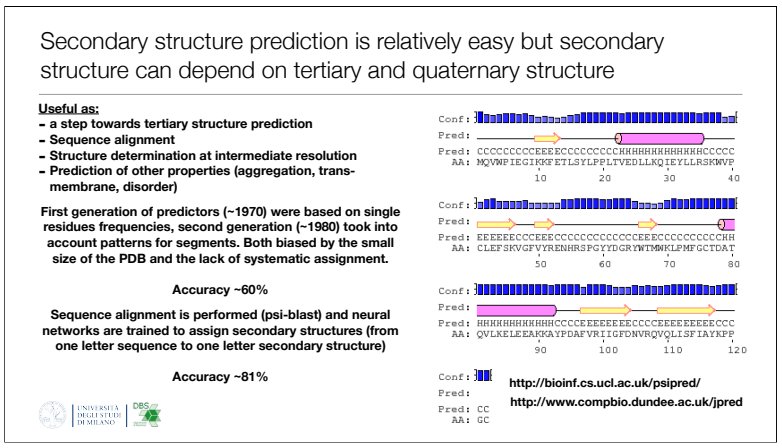



Main idea: sequencing has outpaced structure determination. We have an experimental structure for less than 1 in 1000 known sequences, hence the need for structure prediction.
The databases: the wwPDB (RCSB in the US, PDBe in Europe, PDBj in Japan) handles deposition (OneDep), validation and archiving of more than 200,000 entries. Each entry contains coordinates, experimental data and a validation report (resolution, R-free, Ramachandran outliers, clashscore: see Lecture 1). UniProtKB has two sections: Swiss-Prot (manually reviewed, about half a million entries) and TrEMBL (automatically annotated, hundreds of millions). UniRef clusters sequences at 100%, 90% and 50% identity (UniRef100/90/50) and is used to build MSAs faster and with less redundancy; UniParc is the non-redundant archive of all sequences. AlphaFold-type methods also search large metagenomic databases (e.g. MGnify, BFD) to build deeper MSAs.
Practical tip (T02, Part 1): always start from the UniProt entry of your protein: domains, PTMs, disordered regions, isoforms, and cross-links to the PDB and to the AlphaFold Database.
The gap: UniProt grows in jumps driven by commercial next-generation sequencing (from ~2005) and by metagenomics; the PDB grows roughly linearly, by about 15,000-20,000 entries per year (more than 20,000 deposited in 2025). Note that the two y-axes differ by a factor of 1000. Moreover, many PDB entries are the same protein with different ligands or mutations, so the number of unique sequences with a structure is even smaller. Sequencing a genome takes hours; determining a structure can take months to years.
A second, subtler point: the PDB is also the training set of all machine-learning predictors, and its biases (proteins that crystallise or are amenable to cryo-EM/NMR, mostly well folded, many from model organisms) are inherited by the predictors.

Remember: PDB (www.rcsb.org) = experimental structures; UniProt (www.uniprot.org) = sequences and annotations; the sequence-structure gap is the motivation for computational structure prediction.




Main idea: a protein structure can be represented at different levels of detail, and the choice of representation is a key ingredient of every prediction method.
Explanation: (1) sequence: a 1D string over 20 letters. (2) Secondary structure: a 1D string of H (helix), E (strand), C (coil), assigned from the H-bond pattern of a structure by DSSP. (3) Contact map: an N×N matrix with 1 when the Cβ-Cβ distance (Cα for glycine) is below 8 Å; a distance map stores the actual distances. (4) 3D coordinates: x, y, z of every atom in Å (the ATOM records of a PDB file).
How to read a contact map: an α-helix gives a thick band next to the diagonal (contacts i, i+3/i+4); an antiparallel β-sheet gives a line perpendicular to the diagonal; a parallel β-sheet a line parallel to it.
Why maps are convenient prediction targets: they are invariant to rotations and translations of the molecule (coordinates are not), and a complete distance map determines the 3D structure up to its mirror image.
Remember: maps are 2D, rotation-invariant summaries of a 3D structure; you should be able to recognise helices and sheets in them. Next: the simplest prediction, the 1D secondary structure.

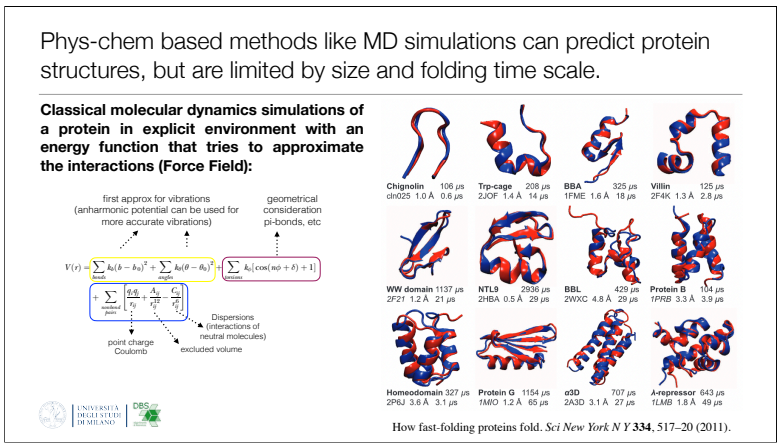



Main idea: predicting the secondary structure (H/E/C for each residue) is relatively easy and useful, but intrinsically limited, because local structure also depends on the long-range context.

Explanation: accuracy is measured as Q3, the fraction of residues assigned to the correct one of three states. First generation (Chou-Fasman, GOR, 1970s): single-residue propensities (e.g. Ala, Leu, Glu favour helices; Val, Ile strands; Pro, Gly break both), about 60%. Second generation: windows of neighbouring residues. Third generation (PSIPRED 1999, JPred): an MSA profile from PSI-BLAST fed to a neural network, about 80%; modern deep-learning methods reach about 85%. The ceiling (~88-90%) exists because the DSSP assignment itself is ambiguous at the ends of elements, and because the same segment can adopt different conformations in different contexts (chameleon sequences, disordered regions that fold upon binding).

Note the recipe: evolutionary information (MSA) + machine learning. AlphaFold scales up exactly this recipe from 1D to 3D. The "Conf" bars in the PSIPRED output are a per-residue confidence: keep them in mind when we discuss pLDDT.

In T02 you use secondary structure prediction to map the components of your complex.



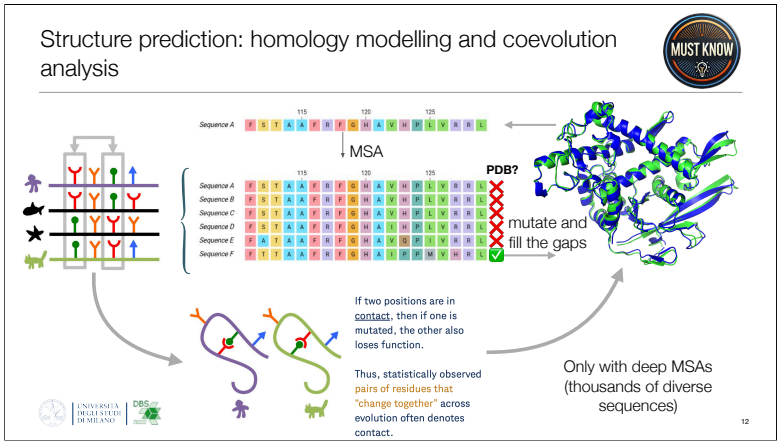

Main idea: in principle, physics-based simulations can predict structures from the sequence alone (Anfinsen), but in practice they are limited to small, fast-folding proteins because of the sampling problem.

Explanation: a classical force field describes the energy as a sum of bonded terms (harmonic bonds and angles, periodic dihedrals) and non-bonded terms (Lennard-Jones for dispersion and excluded volume, Coulomb for point charges); the details are the subject of Lecture 5.

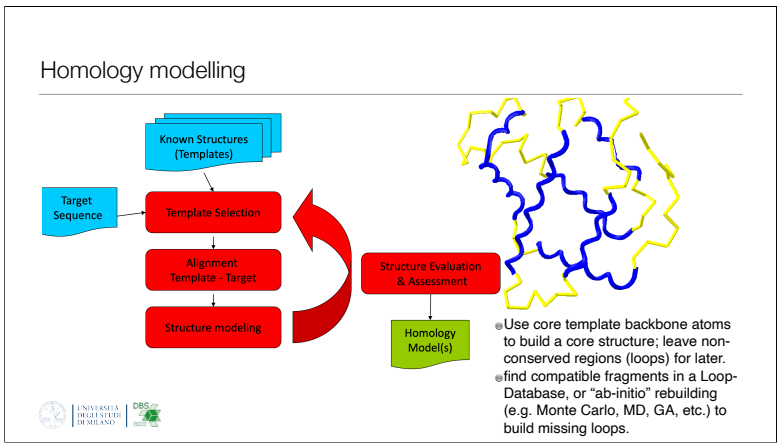
Lindorff-Larsen et al. (Science 2011) used the special-purpose supercomputer Anton to fold 12 small proteins (10-80 residues) with simulations of 100 μs to 1 ms each, reaching structures within a few Å of experiment.

Link to Lecture 2: rates are set by free-energy barriers, and to observe folding (and unfolding, to get the equilibrium populations) you need to simulate many times the folding time. Folding times are μs to s, while a GPU simulates about 1 μs per day for a small protein in water. On top of sampling there is the accuracy of the force field (the scoring problem).

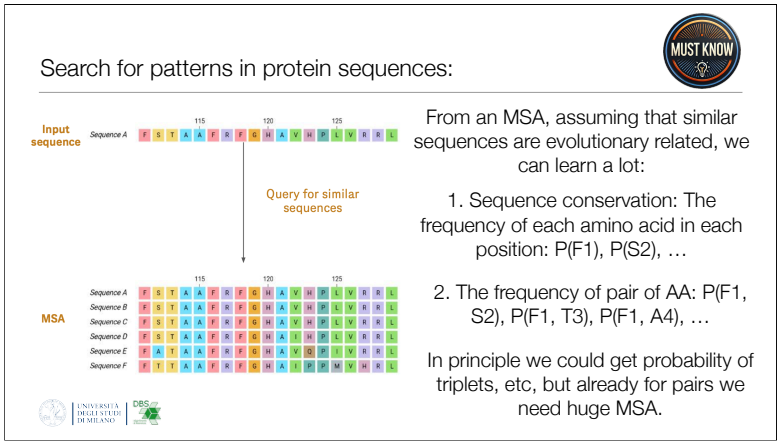
Remember: physics-based prediction is right in principle but limited by sampling (time scales) and by force-field accuracy. This is why we need shortcuts that exploit evolutionary information.



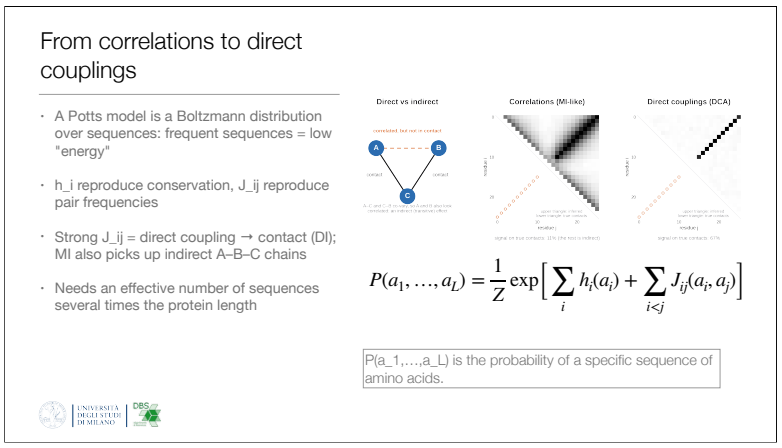

Main idea: two ways to exploit evolutionary information. (1) If a homolog with known structure exists, copy it (homology modelling). (2) If not, use the statistics of many homologous sequences (MSA) to infer which residues are in contact (coevolution analysis).
Explanation: we build an MSA of homologs of the target (sequence A) and check the PDB. If at least one homolog has a structure (✅), we use it as template: mutate the residues and fill the gaps. If none has (❌), we look at correlated mutations: when two residues are in contact, a mutation of one that destabilises the contact is often compensated by a mutation of the other (a salt bridge with swapped charges, a small residue facing a large one). Statistically, columns of the MSA that change together denote contacts.
The requirement is a deep and diverse MSA: the 10K sequences on the slide refer to raw sequences; what matters is the effective number of non-redundant sequences, which for classical coevolution methods had to be several times larger than the protein length.
Remember: template available → homology modelling; no template but deep MSA → coevolution; neither → hard case (the typical situation before AlphaFold2).



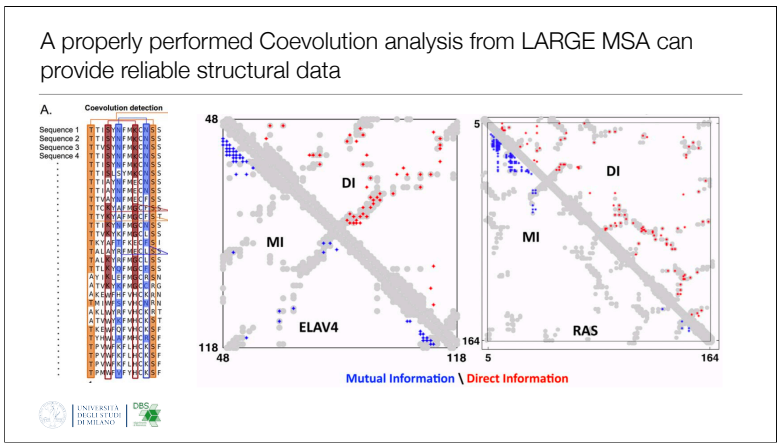

Main idea: homology modelling is a pipeline: template search and selection → target-template alignment → model building → model evaluation, possibly iterated.
Explanation: Templates are searched in the PDB with BLAST or, more sensitively, with profile/HMM methods (HHpred). Selection criteria: sequence identity, coverage, experimental resolution, and the biological state (apo vs holo, active vs inactive, oligomeric state). The alignment is the most critical step: errors cannot be corrected later (a one-residue shift in a helix moves every side chain to the wrong face). Model building: copy the conserved core backbone, place side chains from rotamer libraries, rebuild loops and insertions from loop databases or by ab-initio sampling (Monte Carlo, MD, genetic algorithms): sampling and scoring again, on a small scale. Evaluation: global and local quality estimates (in SWISS-MODEL: GMQE and QMEAN), stereochemistry (Ramachandran plot).
Expected accuracy: above ~50% identity models are close to experimental quality (1-2 Å Cα RMSD); 30-50% the core is right but loops and side chains have errors; below 30% it is risky. Tools: SWISS-MODEL (T02, Part 3), MODELLER. In the figure the conserved core is blue and the remodelled loops are yellow.
Remember: a homology model cannot be better than its template and its alignment, and it inherits the template's state (conformation, bound ligands, oligomer).



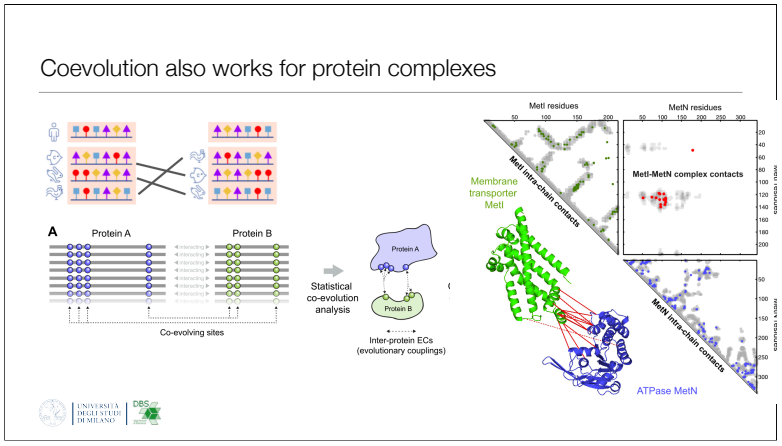

Main idea: an MSA is a statistical sample of sequences compatible with a given fold and function. From it we can estimate single-site frequencies (conservation) and pair frequencies (coevolution).
Explanation (link to Lecture 2: probabilities are estimated by counting, i.e. by histograms): f_i(a) is the fraction of sequences with amino acid a at position i; strongly conserved columns point to residues important for structure or function (catalytic sites, hydrophobic core). f_ij(a,b) is the joint frequency of a at position i and b at position j. If the two positions were independent, f_ij(a,b) = f_i(a)·f_j(b); deviations from this product signal a coupling. A simple measure is the mutual information MI_ij = Σ_ab f_ij(a,b) ln[f_ij(a,b)/(f_i(a) f_j(b))].
The sampling problem: each pair of columns has 20×20 = 400 combinations and there are L(L-1)/2 pairs (about 45,000 for L = 300); triplets would need 8,000 combinations per triplet, hopeless. Moreover sequences are not independent samples: close relatives share mutations by descent, not by coevolution, so sequences are re-weighted (e.g. clustering at 80% identity), giving an effective number of sequences (Neff).
Remember: conservation = one-site statistics; coevolution = two-site statistics; both are counting problems limited by the size and diversity of the MSA.



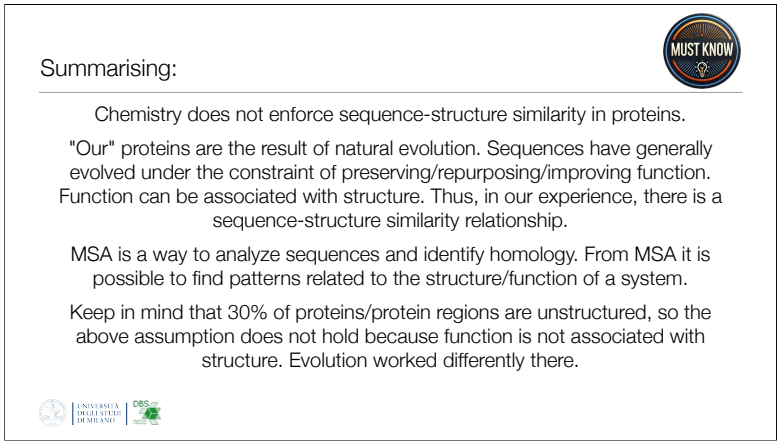

Main idea: to separate direct couplings from indirect correlations we fit a global model of whole sequences, a Potts model, which is a Boltzmann distribution over sequences.
Explanation: P(a_1,...,a_L) = (1/Z) exp[Σ_i h_i(a_i) + Σ_{i<j} J_ij(a_i,a_j)] is the probability of observing the sequence a_1...a_L in the family. Read it as in Lecture 2: an "energy" E(sequence) = -Σh - ΣJ, so sequences that are frequent in the family have low energy. The fields h_i are fitted to reproduce the conservation of each column, the couplings J_ij to reproduce the pair frequencies. Because all pairs are fitted together, a correlation between A and B that is fully explained by A-C and C-B does not need a coupling J_AB: only direct interactions get strong couplings. The strength of J_ij (the direct information, DI) ranks the predicted contacts.
The figure (toy model): left, A and B are correlated only through C; centre, the correlation (MI-like) map shows signal smeared on many pairs, only 11% of it on true contacts; right, the direct couplings concentrate 67% of the signal on true contacts. The real data are on "A properly performed coevolution analysis…" (ELAV4 and RAS).
Requirement: about $L^2 \times 400$ parameters must be fitted, so the MSA must be deep: an effective number of sequences several times the protein length.
Remember: MI = all correlations; DI = direct couplings of a global, Boltzmann-like model of the sequences.



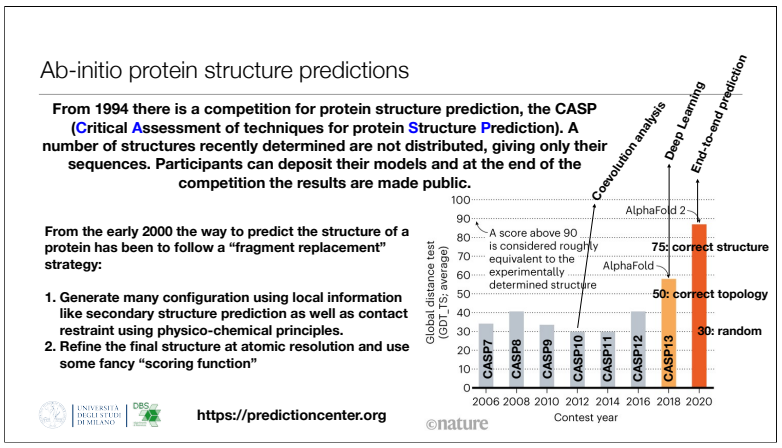

Main idea: on real data, correlations (MI) and direct couplings (DI) give very different contact maps: only DI recovers the native contacts reliably. The model behind DI is on "From correlations to direct couplings".
Explanation: if residue A contacts B and B contacts C, then A and C appear correlated even if they are far apart. Mutual information (MI, lower triangles) mixes direct and indirect correlations and gives many false positives. Direct Coupling Analysis (DCA, Weigt, Morcos and co-workers 2011; EVcouplings, Marks and co-workers 2011) fits a global statistical model of whole sequences, a Potts model:
P(sequence) = (1/Z) exp[ Σ_i h_i(a_i) + Σ_{i<j} J_ij(a_i, a_j) ]
This is a Boltzmann distribution (Lecture 2) with an "energy" E(sequence) = -Σh - ΣJ: sequences that are frequent in the family have low energy. The fields h and couplings J are fitted so that the model reproduces the observed single and pair frequencies; a strong coupling J_ij means a direct interaction, measured by the direct information (DI, upper triangles, red). In the ELAV4 and RAS examples the DI pairs fall on the native contacts (grey) much better than the MI pairs.
The top ~L predicted contacts are usually enough to fold the protein with a structure-building program; this enabled the first de novo predictions of large proteins, including membrane proteins.
Remember: MI = all correlations (direct + indirect); DI = direct couplings from a global, Boltzmann-like model of the sequences.




Main idea: coevolution also works between proteins: residues at an interface coevolve across the two families, which allows predicting interacting partners and interfaces.
Explanation: the MSAs of protein A and protein B are concatenated row by row, pairing sequences from the same organism (ideally orthologs; in bacteria interacting proteins are often encoded in the same operon, which makes pairing easier). Inter-protein couplings then reveal interface contacts: in the ABC transporter example (MetI-MetN) the red inter-protein contacts are consistent with the complex structure.
Difficulties: pairing is ambiguous when there are many paralogs; there are fewer paired sequences than single ones; transient or eukaryote-specific complexes have weaker signals.
This is the principle used by AlphaFold-Multimer and ColabFold (paired + unpaired MSAs) in T02, and the reason why the confidence of predicted interfaces (ipTM) is usually lower than the confidence of each chain.
Remember: intra-chain couplings → fold; inter-chain couplings → interface, provided that the sequences can be correctly paired.

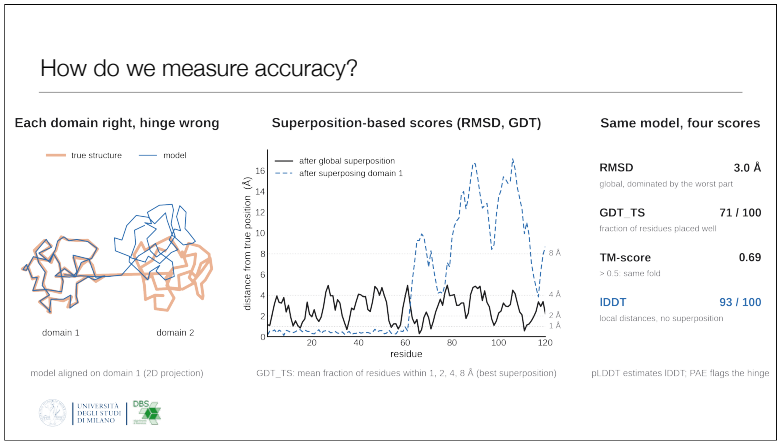



Main idea: recap of the first part. Evolution, not chemistry, is the reason why similar sequences have similar structures; MSAs are the tool to exploit it.
Explanation: (1) chemistry allows very similar sequences with different structures (the GA/GB example); (2) natural proteins evolved under the constraint of preserving or repurposing function, and function is associated with structure; (3) MSAs identify homology and contain structural patterns (conservation, coevolution); (4) intrinsically disordered regions (about 30% of eukaryotic proteomes) are functional without a fixed structure (signalling, short binding motifs): their alignments are poor and carry little coevolution signal, and predictors give them low confidence (the AFDB examples). This connects with Lecture 1: structures are ensembles. Remember, the three questions to ask before any prediction: are there templates? How deep and diverse is the MSA? Is the region likely to be ordered?

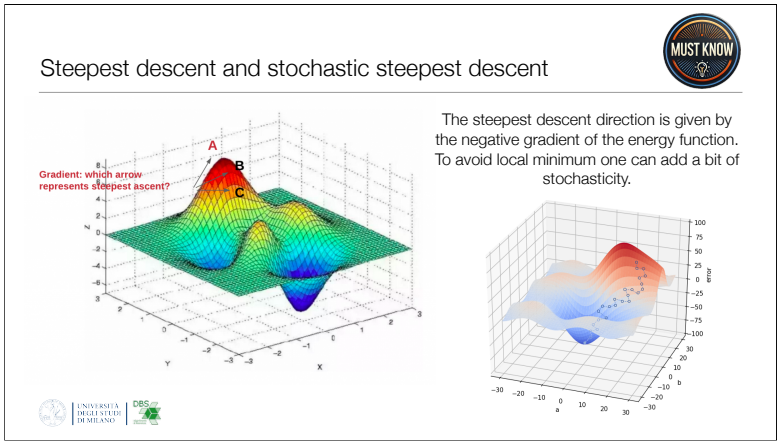



Main idea: CASP is the blind test that has driven the field since 1994. Before deep learning the best "ab initio" methods assembled fragments by Monte Carlo and scored them with knowledge-based energies.
Explanation: CASP runs every two years: targets are structures already solved but not yet released; groups submit models; independent assessors compare them with the experiment. The main score is GDT_TS (defined on "How do we measure accuracy?"). Rule of thumb from the chart: ~30 random, ~50 correct topology, ~75 correct structure, >90 comparable to experiment.
Fragment replacement (Rosetta, Baker lab): 3- and 9-residue fragments are taken from PDB structures with similar local sequence and predicted secondary structure; Monte Carlo moves replace fragments along the chain; models are scored with coarse knowledge-based terms, i.e. statistical potentials obtained by Boltzmann inversion of PDB frequencies, E = -kT ln(P_observed/P_reference) (Lecture 2); the best models are refined at full-atom resolution. Monte Carlo, the sampling engine here, is explained in "Monte Carlo sampling".
The chart: hard-target accuracy was stuck at GDT_TS 30-40 for a decade; jumps came with coevolution (CASP11-12), deep learning (CASP13, AlphaFold1) and end-to-end prediction (CASP14, AlphaFold2).
After the chart: CASP15 (2022) and CASP16 (2024) confirmed that single-domain proteins are essentially solved by AlphaFold-like methods; the focus moved to complexes, nucleic acids, ligands and alternative conformations. For complexes CASP16 showed only moderate progress over CASP15: AlphaFold3-based pipelines with custom MSAs and massive sampling led, top groups predicted slightly more than half of the targets to high accuracy, and antibody-

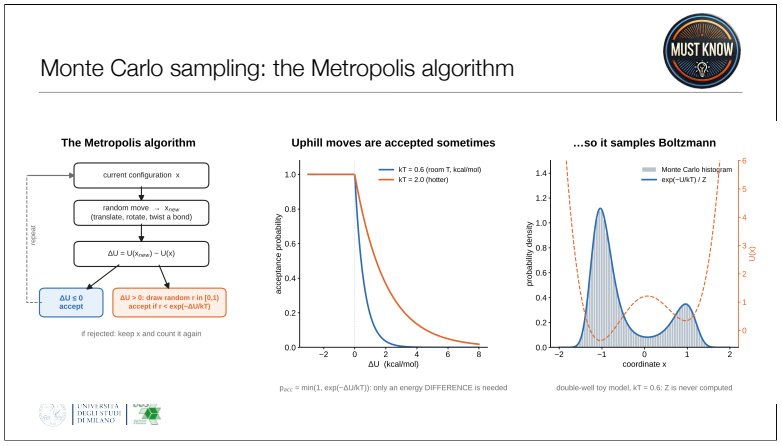
antigen interfaces remained hard.
Remember: CASP = blind benchmark; GDT_TS = % of residues placed correctly; pre-deep-learning = fragment assembly + Monte Carlo + statistical potentials.



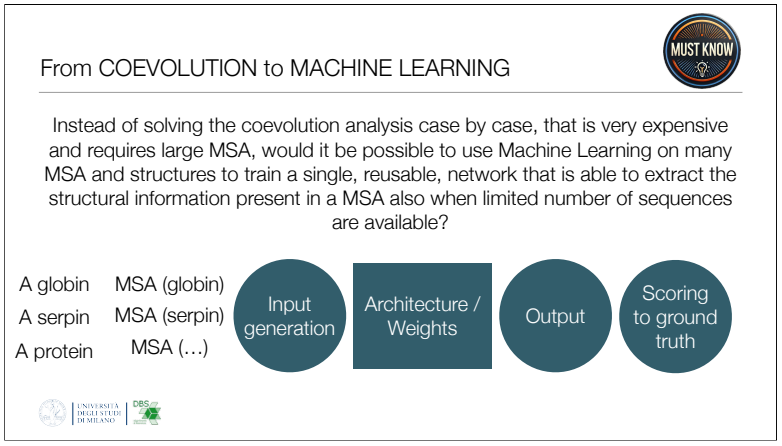

Main idea: the same model can look bad or excellent depending on the score, because each score answers a different question.
Explanation: in the toy two-domain protein each domain is modelled correctly but the hinge between them is wrong. Global superposition spreads the error over the whole chain (black curve); superposing on domain 1 shows that domain 1 is perfect and domain 2 is displaced by up to 16 Å (dashed). The four scores: RMSD 3.0 Å (global, dominated by the worst part); GDT_TS 71/100 (fraction of residues placed well, best superposition at each cut-off of 1, 2, 4, 8 Å); TM-score 0.69 (length-normalised, > 0.5 = same fold; < 0.17 random); lDDT 93/100 (local distances within 15 Å, no superposition, so the hinge error barely matters).
Connection with the confidence metrics: pLDDT predicts lDDT, so the domains would have high pLDDT; the hinge error would appear in the PAE as a high off-diagonal block. For complexes the interface is scored with DockQ (> 0.23 acceptable, > 0.49 medium, > 0.80 high), which you will meet in T02.
Remember: RMSD, GDT_TS and TM-score judge the global arrangement; lDDT judges local accuracy; always ask which one a paper reports.




Main idea: to find the minimum of an energy (or loss) function, move downhill along the negative gradient; adding some randomness helps to escape local minima.
Explanation: the gradient ∇U points in the direction of steepest ascent (question on the figure: the arrow perpendicular to the contour lines, pointing uphill). Steepest descent iterates x_new = x - η ∇U(x), where η is the step size. It converges to the nearest local minimum, so the result depends on the starting point: on a rugged landscape with many minima this is the main limitation. Stochastic variants add noise, use random restarts, or anneal (simulated annealing). In machine learning, stochastic gradient descent (SGD) computes the gradient on random mini-batches of the training set (Lecture 3); the right panel is the loss landscape of a network. The same algorithm appears everywhere in the course: training a network (minimising the loss in weight space), building a structure from a distogram (minimising U in coordinate space), energy minimisation before an MD simulation (Lecture 5), and the local optimisation of docking poses in Vina (see "Monte Carlo sampling").
Remember: gradient descent finds a minimum, not necessarily the global one; sampling strategies (noise, restarts, Monte Carlo) address this.




Main idea: the Metropolis Monte Carlo algorithm generates configurations with the Boltzmann probability without ever computing the partition function, using only relative probabilities.
Explanation: random sampling wastes most of the time on high-energy configurations, and computing P(x) = exp(-U(x)/kT)/Z requires Z, a sum over all configurations. But relative probabilities only need energy differences (Lecture 2: relative probabilities are much easier than absolute ones): P(x_new)/P(x) = exp(-ΔU/kT). Algorithm: (1) from the current configuration x propose a random move (a translation, rotation or torsion change of the ligand); (2) compute ΔU = U(x_new) - U(x); (3) if ΔU ≤ 0 accept; otherwise draw a random number r in [0,1) and accept if r < exp(-ΔU/kT), else keep the old configuration (and count it again); (4) repeat. Downhill moves are always accepted, uphill moves sometimes, with a probability controlled by the temperature kT: this is what lets the search escape local minima. Lowering the temperature gradually (simulated annealing) turns the sampler into an optimiser.
In docking: Vina combines Monte Carlo steps with a local gradient-based optimisation of each pose (BFGS); the exhaustiveness parameter sets the number of independent Monte Carlo runs. Monte Carlo is also the engine of Rosetta fragment assembly (CASP slide) and of loop modelling (homology modelling), and it returns in Lectures 5-6.
Figure (fig_monte_carlo_metropolis): left, the algorithm; centre, the acceptance probability for two temperatures: at kT = 0.6 kcal/mol an uphill move of 2 kcal/mol is accepted ~4% of the time, at kT = 2 ~37%; right, a Monte Carlo run on a double-well potential reproduces the Boltzmann distribution without ever computing Z.
Remember: accept with probability min(1, exp(-ΔU/kT)).

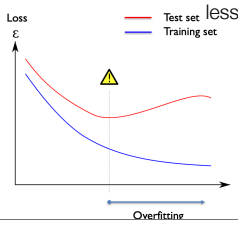



Main idea: instead of fitting a new statistical model for every protein family, train one network on many MSAs and structures, learning the mapping MSA → structure once and for all, and use it also for families with few sequences.
Explanation (vocabulary from Lecture 3): supervised learning. Input generation: features extracted from the MSA (profiles, covariances or couplings); architecture with weights; output (contacts, distances, coordinates); scoring against the ground truth (the experimental PDB structure) through a loss function. Because the network has seen thousands of families, it learns general rules (what a β-sheet contact pattern looks like, which distances are physically compatible) and can "denoise" weak coevolution signals from shallow MSAs.
Training and test sets are separated by date or by sequence similarity, otherwise the network memorises the PDB instead of learning to predict (overfitting, Lecture 3).
Remember: machine learning amortises the coevolution analysis: learn once, apply to every protein.

Not all ML are the same

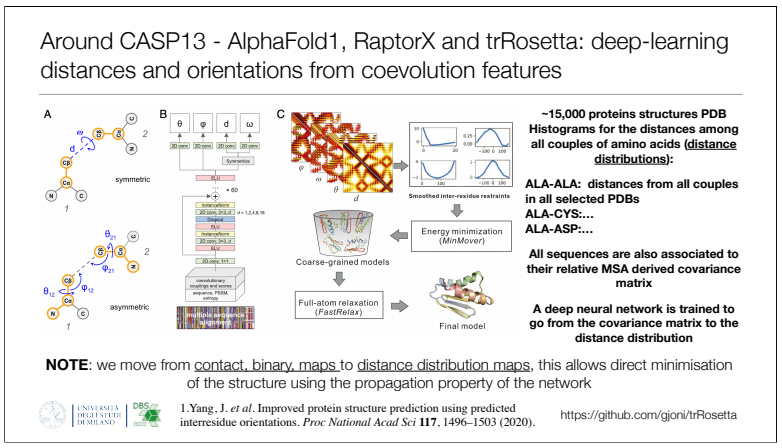
The architecture is the creative, trial-and-error, part. Not all architectures are equally good at learning from a limited amount of data. Architecture design implies making strong hypotheses on how we want to interpret the data.

The weights are obtained for a given architecture and are the result of a more or less costly computer optimisation.



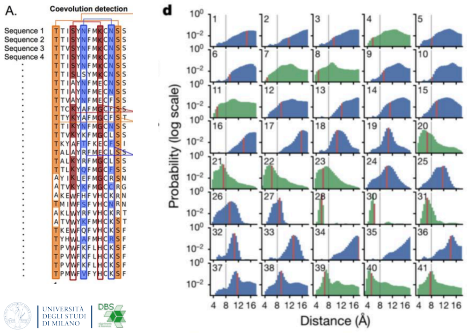

To train an architecture, that means to find the weights, you need a SCORING function that is a measure in some way of the distance of the output from the Training set.
It could be something as simple as sum[(output-training)^2]

Main idea: architecture and weights play different roles. The architecture is a design choice that encodes our hypotheses about the data (inductive bias); the weights are learned by minimising a loss.
Explanation (recap of Lecture 3): the loss on the training set keeps decreasing during training, while the loss on the test set reaches a minimum and then increases: that is overfitting, and training should stop there. The loss is the ML analogue of the energy or score of Lecture 2, and training is an optimisation problem, solved by (stochastic) gradient descent (see "Steepest descent and stochastic steepest descent").
Architecture examples in this lecture: convolutional networks that treat the L×L pair map like an image (AlphaFold1, RaptorX, trRosetta); attention/transformers (AlphaFold2). A good inductive bias lets a network learn from less data: AlphaFold2 builds in that the result must not depend on a global rotation or translation, and that inter-residue distances must be geometrically consistent (triangle updates).
Remember: same data + better architecture → better predictions. The leap of AlphaFold2 was mainly architectural.



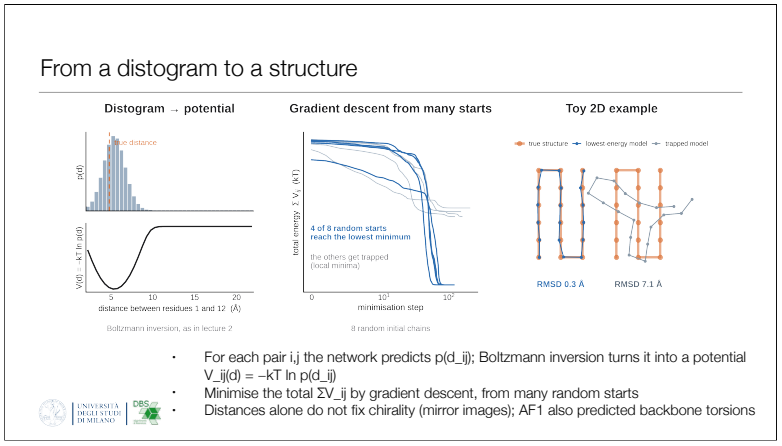

Main idea: around CASP13 (2018) deep networks started to predict, from coevolution features, not just contacts but full distributions of inter-residue distances and orientations, which are then converted into 3D structures.
Explanation: AlphaFold1 and RaptorX introduced distance distributions at CASP13; trRosetta (Yang et al., PNAS 2020), developed shortly after, added inter-residue orientations and made the approach freely available. Input: an MSA, converted into 1D and 2D features (profiles, couplings from a Potts model). A deep residual convolutional network outputs, for every residue pair, the distribution of the Cβ-Cβ distance d (binned from 2 to 20 Å) and of three orientation angles ω, θ, φ (panel A). Training set: ~15,000 PDB chains, the observed distances and angles being the targets. The predicted distributions are converted into smooth restraint potentials, minimised on a coarse-grained model (MinMover) and refined at full-atom level (FastRelax).
The NOTE on the slide is the key concept: moving from binary contacts to probability distributions gives a smooth, differentiable function of the coordinates that can be minimised directly.
Remember: contact (yes/no) → distogram (probability over distances) → potential → minimisation.

Predicting distogram (histogram of distances) from coevolution is the key ingredient of AI approaches

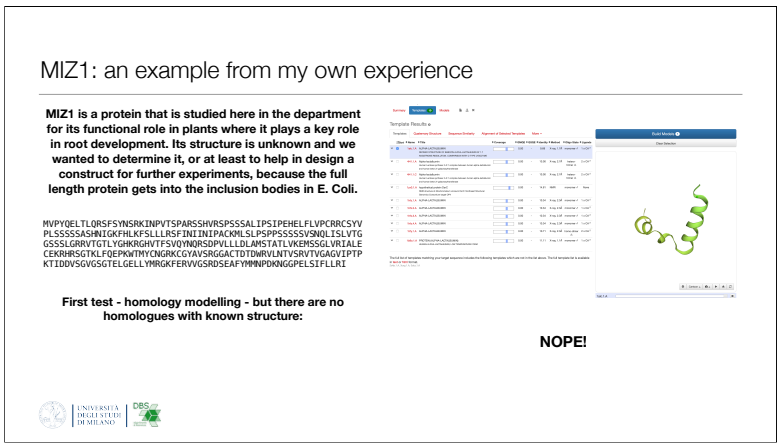



For each pair of atoms (usually Cb) one predict what is the likelihood of finding them at some distance. Then the next step is how to use this information to get a structure.

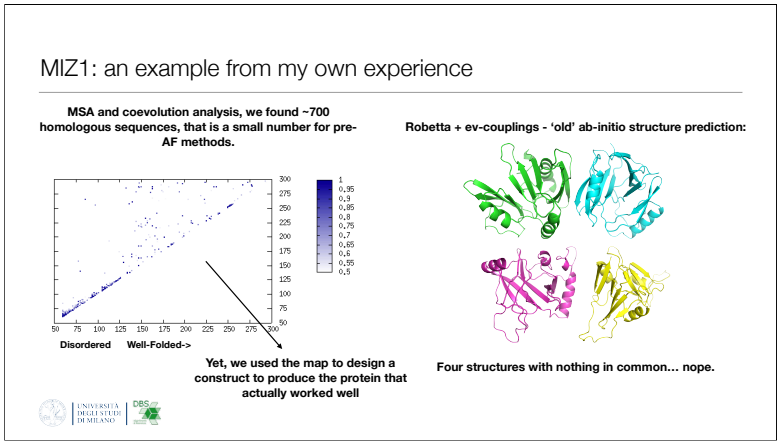
Main idea: a distogram is, for every residue pair, a probability distribution of their distance. It was the key output of pre-AlphaFold2 deep-learning methods.
Explanation: each panel is P(d_ij) for one pair (log scale); the red line is the true distance. A sharp peak means a confident prediction; a broad distribution means uncertainty; probability concentrated at large distances means "not in contact". A distribution is more informative than a single predicted distance because it carries the uncertainty, and possibly more than one peak.
How do we get a structure? With the link between probability and energy from Lecture 2 (Boltzmann inversion): U_ij(d) = -kT ln P_ij(d) (relative to a reference distribution). Summing over all pairs gives a potential U as a function of the coordinates (or of the backbone torsion angles); the most probable structure is its minimum. AlphaFold1 did exactly this, by gradient descent on the backbone torsion angles.
Remember: probability → energy through -kT ln P; structure prediction then becomes an energy minimisation ("From a distogram to a structure").



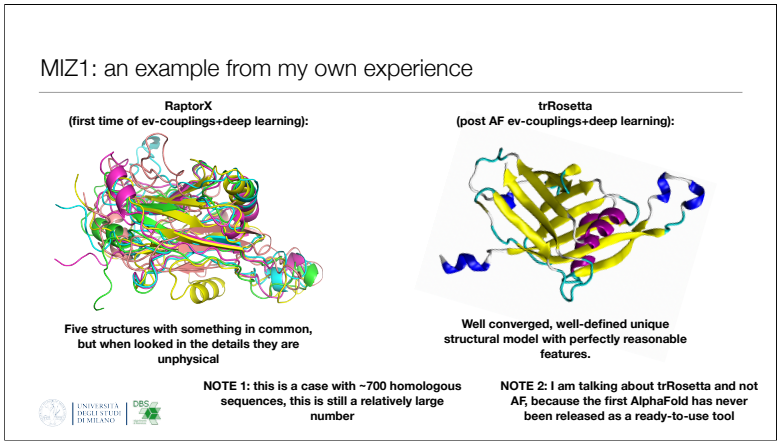

Main idea: how a predicted distogram is turned into a 3D structure: Boltzmann inversion gives a potential, and minimisation gives the structure.
Explanation: this is Lecture 2 applied backwards. There, an energy gives a probability through Boltzmann; here, a predicted probability gives an effective energy, V_ij(d) = -kT ln p(d_ij). A sharp distogram peak becomes a deep, narrow well; an uninformative (flat) distogram gives a flat potential that does not constrain the pair. Summing over all pairs gives a total potential, minimised by gradient descent from many random starting chains.
The toy example: 18 residues in 2D folded only from noisy predicted distograms. Four of eight random starts reach the lowest minimum, within 0.3 Å of the true structure; the other four get trapped in local minima (7.1 Å), which is why we use many starts or add noise (stochastic descent). Distances alone do not fix chirality (a mirror image has the same distances), which is why AlphaFold1 also predicted backbone torsion angles.
This is essentially what AlphaFold did at CASP13 (Senior et al., Nature 2020) and what trRosetta does with Rosetta (Yang et al., PNAS 2020). AlphaFold2 removed this separate step by building the structure inside the network.
Remember: probability → potential (-kT ln p) → minimisation from many starts.



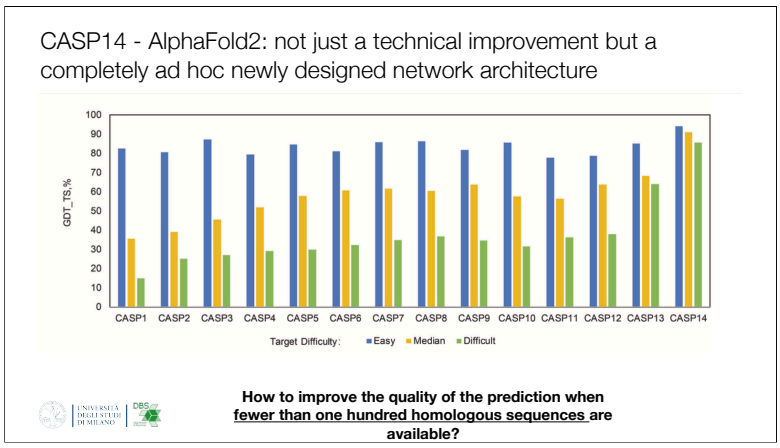

Main idea: a real case study from our department: how predictions for a protein of unknown structure improved over the years, method after method.
Explanation: MIZ1 (MIZU-KUSSEI1) is a plant protein required for root hydrotropism, the growth of roots towards water. Its structure is unknown, and the full-length protein is insoluble when expressed in E. coli (inclusion bodies), so structural information was needed to design a soluble construct for experiments.
First attempt: homology modelling. The template search (SWISS-MODEL) finds no homolog with known structure covering the protein, only a short helical fragment. This is the "no template" case of the homology modelling / coevolution overview.
Question for the students: what would you try next, and what would you need?



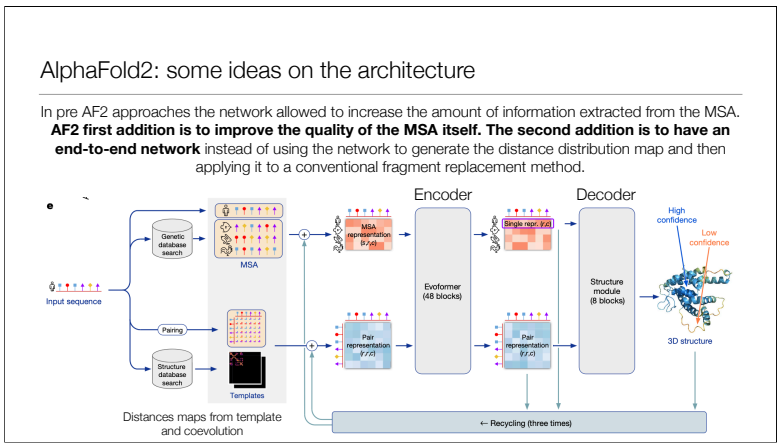

Main idea: with no templates, the next step is coevolution: deep enough MSAs give contacts; old ab-initio methods could not turn them into a reliable structure, but the contact map was already useful.
Explanation: about 700 homologous sequences were found, a small number for pre-AlphaFold methods. The predicted contact map (left) has contacts only in the C-terminal part: the region with contacts is likely to be the folded domain, the region without contacts is likely disordered.
Robetta (Rosetta fragment assembly + evolutionary couplings) produced four models with nothing in common: lack of convergence is itself a warning sign, the method is not confident.
Lesson 1: agreement between independent models is a confidence indicator (the same idea returns with the five ColabFold models).
Lesson 2: even an imperfect prediction can be experimentally useful: the folded/disordered boundary from the contact map was used to design a construct that could be produced well.




Main idea: deep learning on coevolution features turned the same shallow MSA into a convergent, physically reasonable model.
Explanation: RaptorX (one of the first methods combining evolutionary couplings and deep learning) gave five models with a common topology but unphysical details. trRosetta (post-AlphaFold1 approach, see the distogram slides) gave a well converged, unique model with reasonable features. With ~700 sequences, a shallow MSA for classical coevolution, the network extracted a coherent signal because it had learned general structural rules from the PDB.
NOTE 2 on the slide: AlphaFold1 was never released as a ready-to-use tool, which is why trRosetta was the state of the art available to users at that time.
Exercise: run MIZ1 (UniProt: Arabidopsis MIZ1) on ColabFold or check the AlphaFold Database and compare with these models. Is the C-terminal domain predicted with high pLDDT? Does the N-terminal region look disordered?

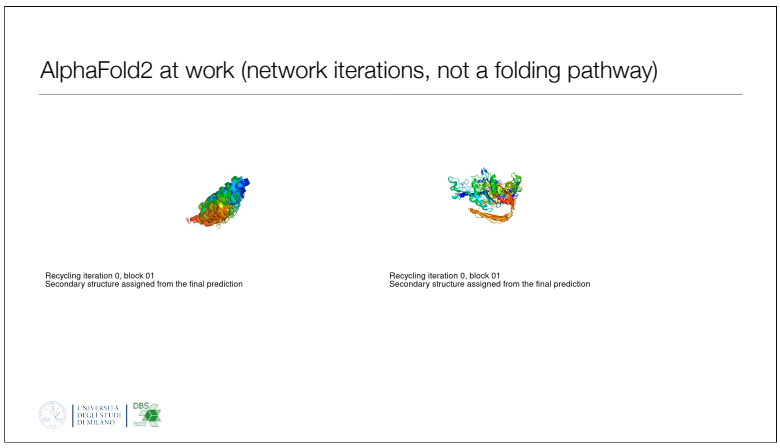



Main idea: CASP14 (2020): AlphaFold2 reached a median GDT_TS around 90, comparable to experimental accuracy for most targets, including the difficult ones, thanks to a newly designed architecture rather than incremental improvements.
Explanation: the bars show the best GDT_TS for easy (blue), medium (yellow) and difficult (green) targets over all CASPs: the green bar at CASP14 is the real revolution, the hard targets without templates. The question at the bottom is the motivation for the architecture on the next slides: how to improve predictions when fewer than ~100 homologous sequences are available? Note that AlphaFold2 still depends on the MSA: its accuracy drops when the effective number of sequences is very small (below ~30), see Jumper et al., Nature 2021.
Context: the 2024 Nobel Prize in Chemistry was awarded to D. Hassabis and J. Jumper (AlphaFold2) and to D. Baker (computational protein design, Lecture 9).
Remember: AlphaFold2 solved single-chain structure prediction for most proteins with a reasonable MSA; the MSA is still the main input.

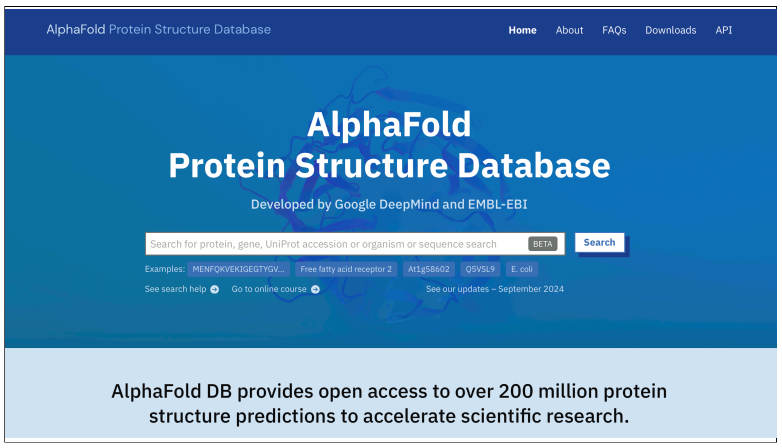



Main idea: AlphaFold2 combines two ideas: it refines the information extracted from the MSA iteratively inside the network, and it is end-to-end, i.e. it outputs atomic coordinates directly instead of a distogram to be fed to a separate folding program.
Explanation: inputs: the query sequence, an MSA from large genetic databases (UniRef90, BFD, MGnify), and templates from the PDB. Two internal representations are built: the MSA representation (sequences × residues) and the pair representation (residues × residues, conceptually a learned "distance map"). The Evoformer (48 blocks) plays the role of the encoder, the structure module (8 blocks) the role of the decoder, producing the 3D structure with a per-residue confidence. The whole network is trained with a loss computed on the final structure (FAPE, frame-aligned point error), plus auxiliary losses (distogram, confidence).
Clarification of the text on the slide: "improving the quality of the MSA" means improving the network's internal representation of the MSA (which sequences and positions are informative), not re-aligning or deleting sequences.
Inside the three stages. Evoformer: in the MSA representation, row-wise attention lets the residues of a sequence exchange information (biased by the pair representation); column-wise attention lets the sequences exchange information at each position, effectively learning how to weight the MSA. The "outer product mean" transfers information from the MSA to the pair representation: this is the learned analogue of the covariance/coevolution analysis of "Search for patterns" and "From correlations to direct couplings". Triangle updates and triangle attention on the pair representation enforce geometric consistency: if i is close to j and j is close to k, this constrains the i-k distance (triangle inequality).
Structure module: each residue is treated as a rigid body (a frame: rotation + translation, a

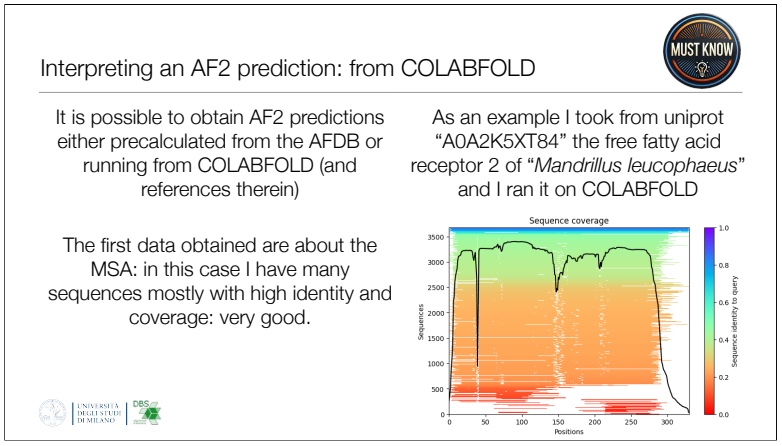
"residue gas"); Invariant Point Attention moves the frames in 3D in a way that does not depend on the global orientation; side-chain torsions are then predicted, giving all-atom coordinates.
Recycling (three times): the predicted structure and representations are fed back as inputs, and the prediction is refined.
Training: PDB structures (cut-off 2018) plus self-distillation on ~350,000 high-confidence predictions of UniProt sequences.
Remember (details): outer product mean ≈ learned coevolution; triangle updates ≈ geometry; recycling ≈ iterative refinement.
Remember: AlphaFold2 = MSA + pair representation, refined jointly, decoded directly into coordinates, trained end-to-end.



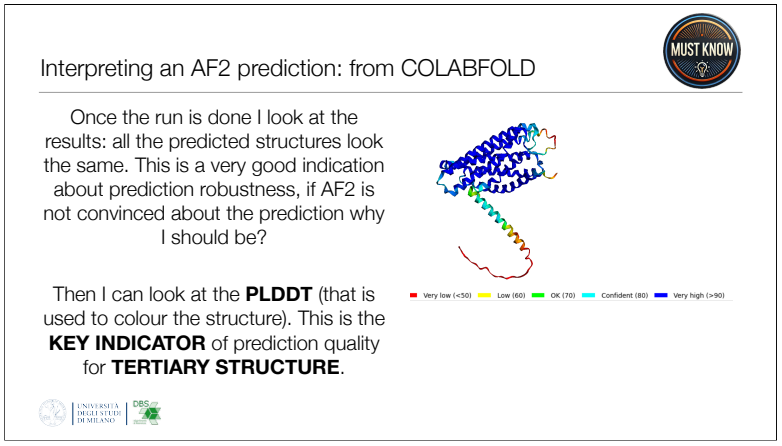

Main idea: watching the prediction emerge across the network layers and recycling iterations (videos from the supplementary material of Jumper et al., Nature 2021).
Explanation: each frame is the structure decoded from the internal representation after a given Evoformer block and recycling iteration; the colour is the confidence (pLDDT). The global fold appears early for easy targets (left), while for difficult targets (right) the structure keeps rearranging for many blocks before converging.
Important misconception: this is NOT a folding pathway. The trajectory has no physical time and no relation to how the real protein folds: it is the network refining its guess. For folding mechanisms and dynamics we need physics-based methods (Lectures 2, 5, 6).



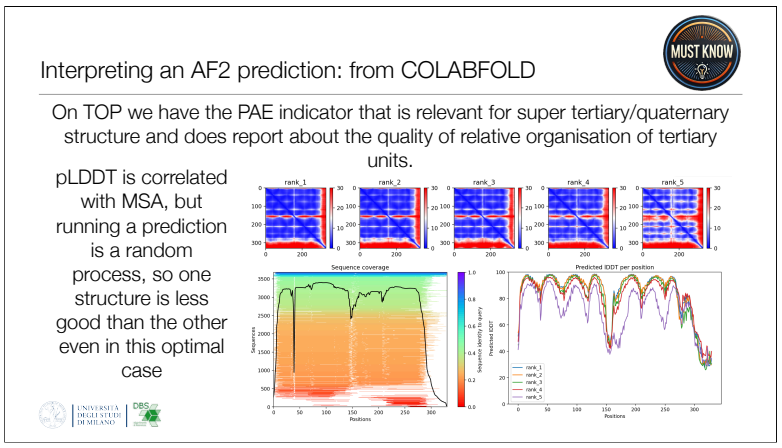

Main idea: the AlphaFold Protein Structure Database (AFDB, Google DeepMind and EMBL-EBI) provides precomputed AlphaFold2 predictions for over 200 million UniProt sequences. Explanation: you can search by UniProt accession, gene name or sequence; each entry provides the model (PDB/mmCIF), the per-residue pLDDT (stored in the B-factor column) and the PAE matrix. Limitations to remember: AFDB entries are single chains (monomers); there are no ligands, cofactors, ions or post-translational modifications; the prediction refers to a specific UniProt sequence version and is not re-run when the MSA databases grow. Practical tip (T02): always check the AFDB first; it tells you in seconds which parts of your protein are confidently folded and which are probably disordered.




Main idea: how to read an AlphaFold2 prediction, starting from the MSA, using ColabFold.
Explanation: ColabFold (Mirdita et al., Nature Methods 2022) runs AlphaFold2 in Google Colab, with a fast MSA search (MMseqs2 server) instead of the original, much slower database search; it is what you use in T02. Example: the free fatty acid receptor 2 (a GPCR) of Mandrillus leucophaeus, UniProt A0A2K5XT84.
Reading the sequence coverage plot: x = residue position; y = aligned sequences, sorted by identity to the query; colour = sequence identity; black line = number of sequences aligned at each position (coverage). Here: more than 3000 sequences, many with high identity and coverage: a very good MSA. Positions with a dip in coverage (termini, some loops) have little evolutionary information, and we should expect lower confidence there.
Remember: first look at the MSA: its depth and coverage tell you in advance where the prediction can be trusted.




Main idea: two checks on the predicted models: agreement between the five models, and the per-residue confidence pLDDT.
Explanation: ColabFold/AlphaFold2 run five networks with different trained parameters, giving five models ranked by confidence. If all five look the same, the prediction is robust: if AlphaFold is not convinced, why should you be?
pLDDT is the network's prediction of the lDDT-Cα score of each residue (0-100). lDDT (local Distance Difference Test) measures, without any superposition, the fraction of inter-atomic distances within 15 Å that are preserved within 0.5, 1, 2 and 4 Å: it evaluates the local environment of each residue. Scale: >90 very high (backbone and most side chains reliable); 70-90 confident (backbone reliable); 50-70 low; <50 very low (often disordered regions, or simply not predictable).
pLDDT is written in the B-factor column of the PDB file, so you can colour by B-factor in ChimeraX or VMD (Lecture 1, T01).
Caution: pLDDT is a confidence, not a measure of flexibility, even if low pLDDT often correlates with disorder.
Remember: pLDDT is the key indicator for the local (tertiary) structure.



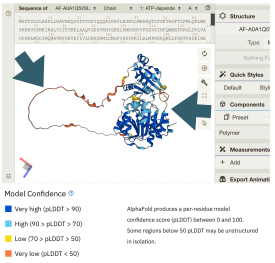

Main idea: PAE (predicted aligned error) tells us how confident the network is about the relative position of different parts of the structure: domains and chains.
Explanation: the PAE matrix is L×L; the entry (x, y) is the expected error, in Å, in the position of residue x when the predicted and true structures are aligned on residue y. Low-error (blue) blocks on the diagonal are domains that are well defined internally; low off-diagonal blocks mean that the relative placement of two domains (or two chains) is confident; high values (red) mean that the relative placement is unknown, e.g. two domains connected by a flexible linker. The matrix is not symmetric.
Global scores derived from PAE: pTM, the predicted TM-score of the whole model, and, for complexes, ipTM, the predicted TM-score of the interfaces (rule of thumb: ipTM > 0.8 confident, < 0.6 likely wrong, in between a grey zone). These are the scores used in T02.
The pLDDT plot of the five models shows that even in this optimal case the models are not identical: the network includes stochastic elements (dropout, different parameters, MSA sampling), which is why ColabFold ranks the models.
Remember: pLDDT → local accuracy; PAE / pTM / ipTM → relative arrangement of domains and chains.

Interpreting an AF2 prediction: from AFDB

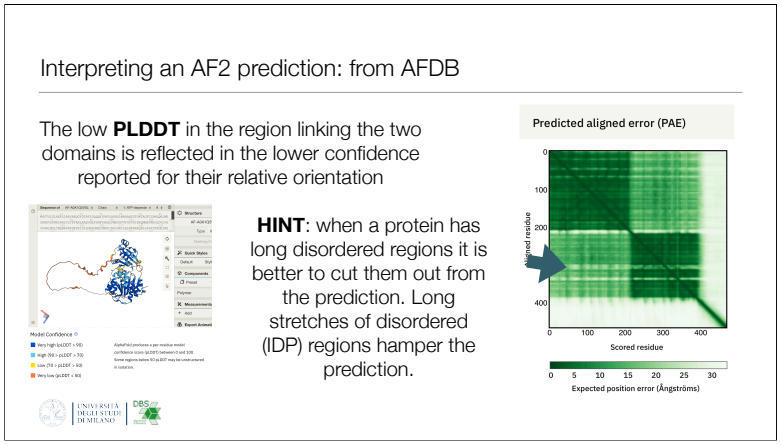
AFDB does not provide information about the MSA and the consistency of the prediction: so we need to FOCUS on **PLDDT** and PAE

Let's look at "Q5VSL9" from AFDB that is a prediction for the "ATP-dependent RNA helicase RhlE" from "*Aeromonas allosaccharophila*":

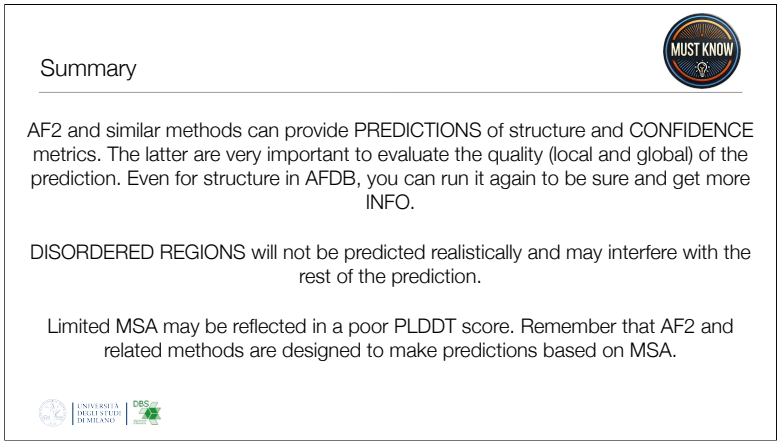
**PLDDT is HIGH** everywhere but in the DISORDERED REGIONS



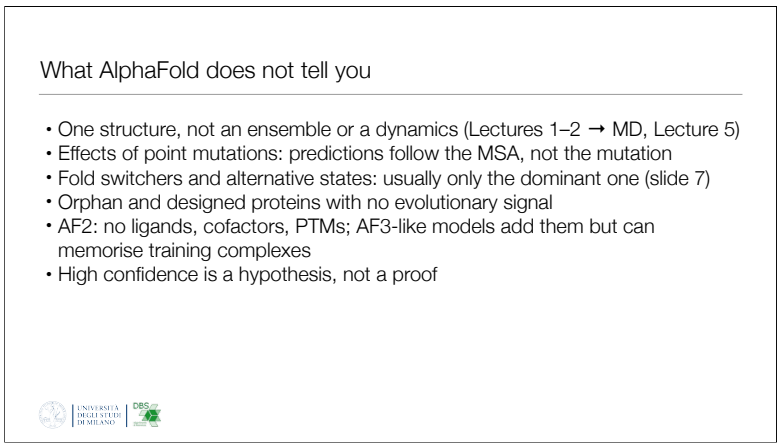

Main idea: when using the AlphaFold Database we do not see the MSA and the five models, so we must rely on pLDDT and PAE.
Explanation: example Q5VSL9 from the AFDB, the ATP-dependent RNA helicase RhlE of Aeromonas allosaccharophila. pLDDT is high everywhere except in the disordered regions (orange, pLDDT < 50). These regions are drawn as long, extended ribbons around the folded domains: those coordinates are not a structure, do not interpret them (the chain is not really extended in solution, it is simply not predicted). In ChimeraX you can hide or colour these regions by pLDDT.
Remember: low pLDDT regions must not be interpreted structurally; they are often, but not always, disordered.



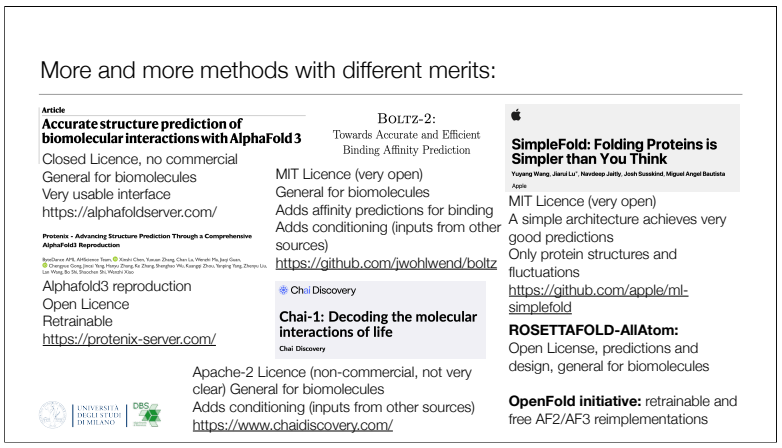

Main idea: low confidence in the linker is reflected in the PAE: the two domains are individually well predicted, but their relative orientation is not.
Explanation: in the PAE plot the two diagonal blocks are dark (low error), while the off-diagonal blocks are lighter (high error): the relative orientation of the two domains is uncertain, and they could rotate with respect to each other.
HINT on the slide: long disordered regions are better cut out of the prediction, because they add noise and can hamper the prediction of the folded part. pLDDT and PAE are also a practical way to define domain boundaries when designing constructs for experiments (exactly what we did for MIZ1).
Caution: some disordered regions fold upon binding a partner, and AlphaFold sometimes predicts the bound conformation with high confidence ("conditional folding"): confidence does not tell you in which state the protein is.
Remember: always read pLDDT and PAE together.




Main idea: summary of structure prediction with AlphaFold-like methods: predictions come with confidence metrics, and the metrics are as important as the structure.
Explanation: points on the slide: (1) always read the confidence metrics (pLDDT local, PAE/ pTM/ipTM global); for AFDB entries you can re-run the prediction to see the MSA and the five models; (2) disordered regions are not predicted realistically and can interfere with the rest; (3) a limited MSA is often reflected in a low pLDDT.
What these methods do NOT tell you is on "What AlphaFold does not tell you".
Remember: prediction + confidence + critical judgement.




Main idea: what AlphaFold-like predictions do not tell you, even when the confidence is high.
Explanation: (1) one structure, not an ensemble and not dynamics (Lectures 1 and 2): alternative conformations require MD (Lecture 5) or tricks such as MSA subsampling; (2) the effect of point mutations: the prediction is dominated by the MSA and is largely insensitive to single mutations, so it cannot predict destabilisation; (3) fold-switching proteins and alternative states: usually only the dominant fold is predicted (the GA/GB example); (4) orphan proteins without homologs and designed proteins have little evolutionary signal; (5) AlphaFold2 has no ligands, cofactors or PTMs; AlphaFold3-like models add them, but can memorise complexes seen in training; (6) high confidence is not proof: a prediction is a hypothesis to be tested.
Remember: a prediction answers "what is the most likely structure, given evolution and the PDB?", not "what does this protein do in this condition?".



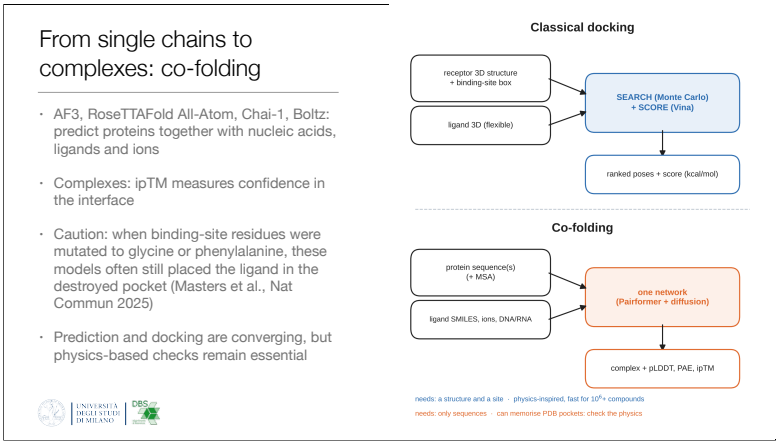

Main idea: after AlphaFold2 many methods appeared, now covering all biomolecules (proteins, nucleic acids, small molecules, ions, modifications), with different merits and licences.

Explanation: AlphaFold3 (Abramson et al., Nature 2024) replaces the Evoformer with the lighter Pairformer (much less MSA processing) and the structure module with a diffusion module that generates all-atom coordinates (diffusion models: Lecture 3). It is available through the AlphaFold Server; the code is available for non-commercial use, with weights on request. Open alternatives: Protenix (a reproduction of AlphaFold3, retrainable), Boltz-1 and Boltz-2 (MIT licence; Boltz-2 adds binding-affinity prediction and conditioning on additional inputs), Chai-1, RoseTTAFold All-Atom (prediction and design), the OpenFold initiative (fully open and retrainable reimplementations, e.g. OpenFold3). SimpleFold (Apple, 2025) shows that a general transformer trained with flow matching, without MSA-specific or triangle modules, can also predict structures well, and can generate structural fluctuations.

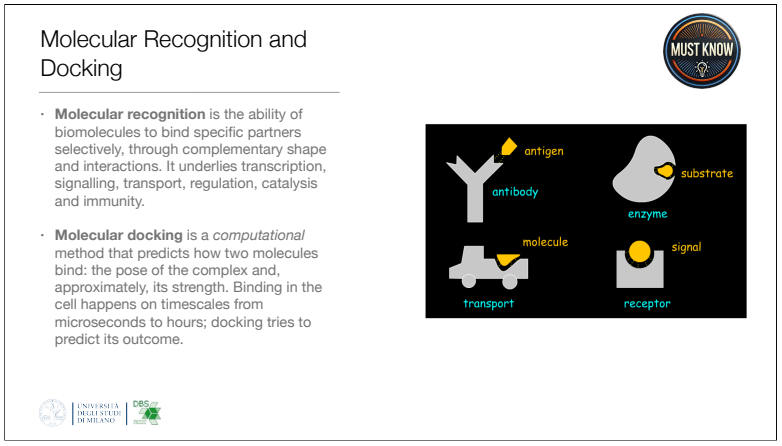
Practical points: licences and versions change quickly, check them before use (especially for commercial work). Independent benchmarks indicate that AlphaFold3 is still marginally the most accurate in most categories, with open models close behind, and all of them weaker on antibody-antigen interfaces. Generative (diffusion) models can "hallucinate" plausible-looking structure in disordered regions: always check the confidence.

In T02 you compare ColabFold (AlphaFold2-Multimer), the AlphaFold3 server and Boltz on the same complex.

Bridge: these methods also predict protein-ligand complexes, which brings us to molecular recognition and docking.

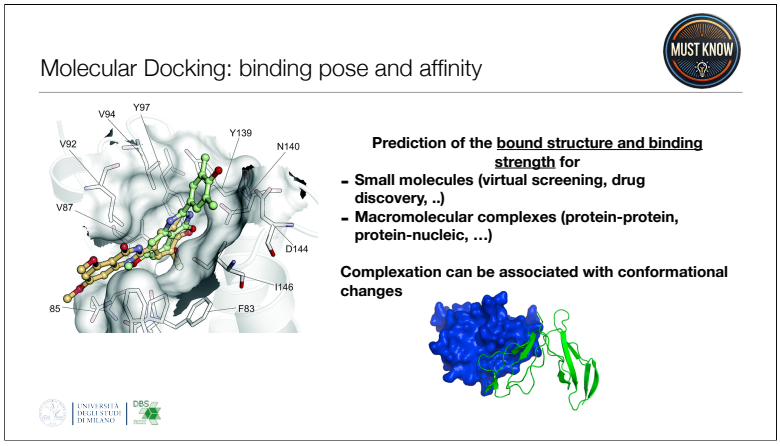



Main idea: co-folding methods predict a complex (proteins, nucleic acids, ligands, ions) directly from sequences and SMILES, so structure prediction and docking are converging; but they learn patterns from the PDB, not physics.
Explanation: AlphaFold3, RoseTTAFold All-Atom, Chai-1, Boltz (and OpenFold3, Protenix) take sequences and ligand SMILES and output the complex with pLDDT, PAE and ipTM (the predicted TM-score of the interfaces: $> 0.8$ confident, $< 0.6$ likely wrong). Classical docking instead needs a receptor structure and a binding site, then searches and scores poses (figure: fig_docking_vs_cofolding).
Caution (Masters et al., Nat Commun 2025): when the binding-site residues were mutated to glycine (removing the interactions) or to phenylalanine (filling the pocket), the models often still placed the ligand in the destroyed pocket: they recognise the protein family and reproduce the pose they have seen, rather than reasoning about interactions. Physical checks (clashes, bond geometry, e.g. PoseBusters) and controls remain essential.
State of the art (CASP16, 2024): for complexes, AlphaFold3-based pipelines with custom MSAs and massive sampling led, but progress over CASP15 was moderate; antibody-antigen interfaces remained hard, and the best group there used docking rather than co-folding.
In T02 you compare ColabFold, the AlphaFold3 server and Boltz on the same complex, with a negative control.
Bridge: this brings us to molecular recognition and docking.

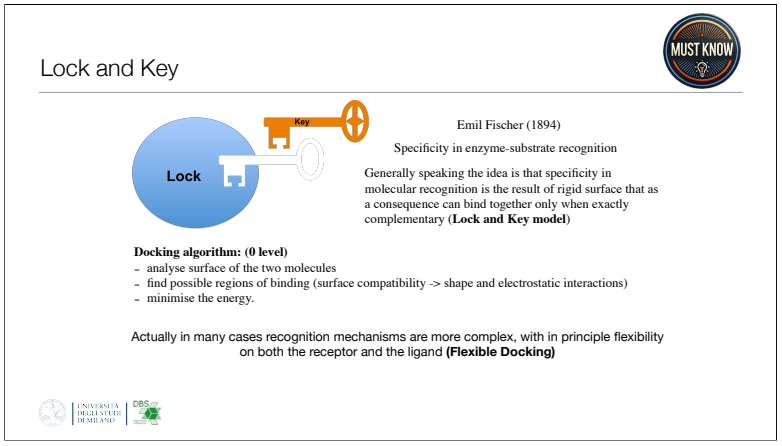



Main idea: molecular recognition is the biological phenomenon; molecular docking is the computational method that predicts it.
Explanation: molecular recognition is the ability of biomolecules to recognise other molecules and interact selectively with them: antibody-antigen, enzyme-substrate, transporter-cargo, receptor-signal. It underlies transcription, translation, signal transduction, transport, regulation, catalysis, infection and the immune response. Molecular docking is the computational prediction of the structure of the complex (and of its binding strength) from the structures of the partners. (The last sentence on the slide refers to the natural binding process, which happens on time scales of microseconds to seconds in a cell; docking itself is a computation.)
Link to Lecture 2: binding is a two-state equilibrium $R + L \rightleftharpoons RL$ with $K_D = k_{off}/k_{on}$ and

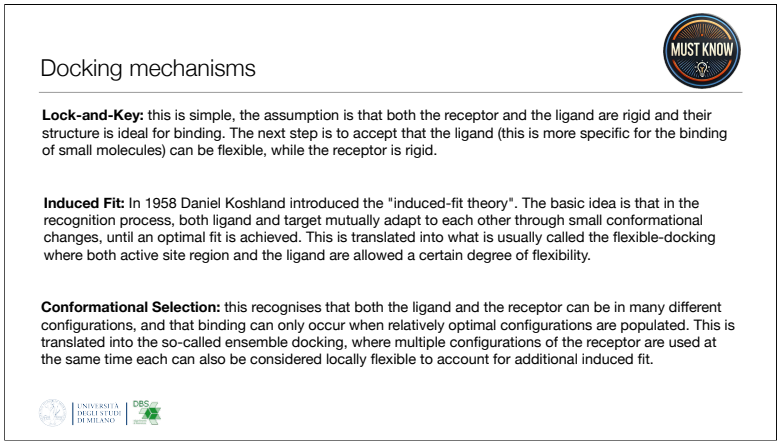
$\Delta G^\circ_{bind} = RT \ln(K_D/1\ M)$. At 298 K, $RT \ln 10 \approx 1.36$ kcal/mol: each factor of 10 in $K_D$ is 1.36 kcal/mol. A $K_D$ of 1 μM corresponds to about -8.2 kcal/mol, 1 nM to about -12.3 kcal/mol. The conversion is on "From $K_D$ to ΔG to a docking score".
Remember: recognition = nature; docking = model. The difference is an exam question.




Main idea: docking aims at two outputs: the binding pose (the geometry of the complex) and the binding affinity (how strong it is).
Explanation: the pose is evaluated against an experimental structure by the RMSD of the ligand atoms; RMSD < 2 Å is conventionally a success. The affinity is estimated by the score, a proxy for ΔG_bind. Pose prediction works much better than affinity prediction. Two families of problems: small molecules (virtual screening, drug discovery), and macromolecular complexes (protein-protein, protein-nucleic acid), with larger, flatter interfaces where shape complementarity and electrostatics dominate. In both cases binding can be associated with conformational changes (bottom: a domain rearranging upon binding), which is the hardest part to model.
Remember: pose (geometry, RMSD) ≠ affinity (energy, ΔG); docking is good at the first, approximate at the second.

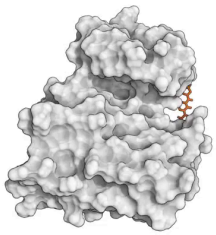



Main idea: the simplest model of molecular recognition is Fischer's lock and key (1894): rigid, exactly complementary surfaces explain specificity.
Explanation: the "level 0" docking algorithm follows directly: analyse the surfaces of the two molecules, find regions that are complementary in shape and electrostatics, then minimise the energy of the complex. For protein-protein docking this is still the first step of many programs: rigid-body search, often with Fast Fourier Transform methods that test billions of relative orientations on a grid (ZDOCK, ClusPro).
Limitation: proteins and ligands are flexible (Lectures 1 and 2: structures are ensembles), so recognition is in general more complex and requires flexible docking (next slide).
Remember: lock and key = rigid docking; good starting point, wrong in general.




Main idea: three models of molecular recognition, each corresponding to a docking strategy.
Explanation: lock and key → rigid docking (or a flexible ligand in a rigid receptor, the most common setting for small molecules, as in T03). Induced fit (Koshland 1958): ligand and receptor adapt to each other through small conformational changes → flexible docking, with flexible ligand and some flexible receptor side chains in the binding site. Conformational selection: the receptor (and the ligand) populate many conformations according to their Boltzmann weights (Lecture 2); the ligand binds the compatible ones and shifts the populations → ensemble docking, using multiple receptor structures (different crystal structures, NMR ensembles, snapshots from MD simulations, predicted models), each possibly with local flexibility to account for some induced fit. In reality the two mechanisms usually coexist.
Link to Lecture 5: MD simulations are a common way to generate receptor ensembles for docking.
Remember: lock and key → rigid; induced fit → flexible; conformational selection → ensemble docking.

MD simulations

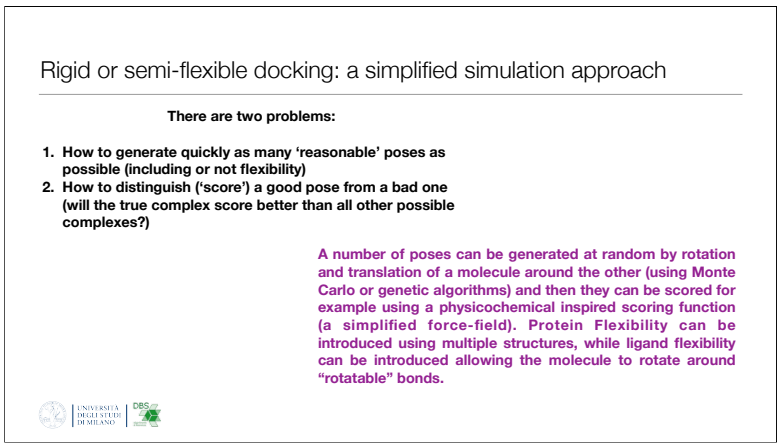


In principle MD simulations can be used to search for ligand binding, but there are problems:

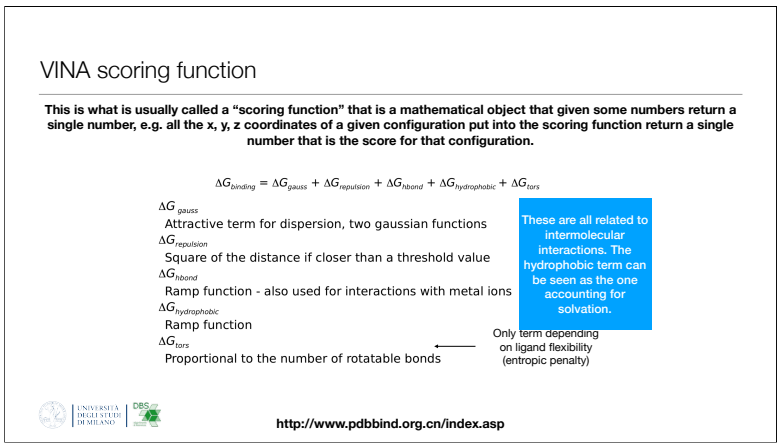
1) Parametrizing force-fields for all the possible chemicals is hard (this is more relevant for ligand binding than for protein complexes)
2) The binding time scale can be slow in particular when associated with conformational changes (the problem of sampling). This is even more relevant for protein complexes where the systems become large and so the simulations become slow.

1.Shan, Y., Kim, E. T., Eastwood, M. P., Dror, R. O., Seeliger, M. A. & Shaw, D. E. How does a drug molecule find its target binding site? *J Am Chem Soc* **133,** 9181–3 (2011).

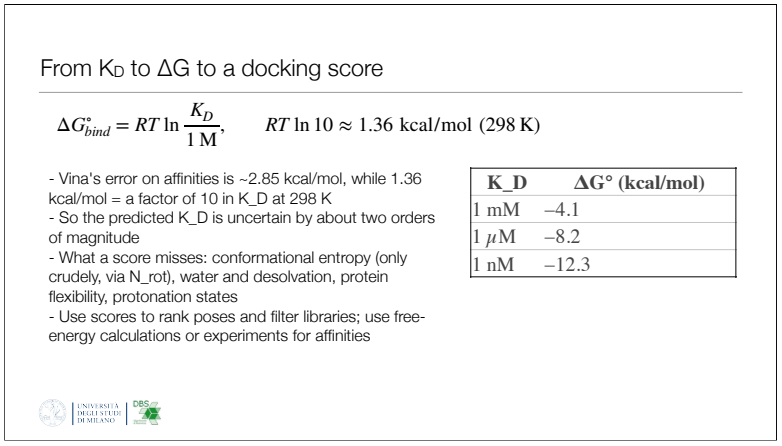
Main idea: MD simulations can, in principle, reproduce the whole binding process, but they face the same two problems we met for folding: force-field parametrisation and time scales.
Explanation: Shan et al. (JACS 2011, D. E. Shaw Research) simulated small kinase inhibitors (including the drug dasatinib) diffusing around Src kinase: the ligands found the binding pocket spontaneously within microseconds and adopted the crystallographic pose (the video).
Problems: (1) parametrising force fields for arbitrary chemicals is hard (generalised force fields such as GAFF or CGenFF help, but are approximate); this matters more for ligands than for proteins, which are made of 20 building blocks; (2) time scales: association can be fast, but dissociation (residence times of seconds to hours, i.e. k_off from Lecture 2) and binding coupled to conformational changes are far beyond plain MD, especially for large protein complexes.
Enhanced sampling (Lecture 6) and free-energy methods provide accurate affinities at a high cost. In practice docking is the fast first step and MD the refinement and validation step.
Remember: MD = physics with explicit dynamics, accurate but expensive; docking = fast approximations.




Main idea: docking is a simplified simulation that has to solve the same two problems of the whole course: SEARCH (sampling the poses) and SCORING (ranking them).
Explanation: the degrees of freedom of a flexible ligand in a rigid receptor are 3 translations + 3 rotations + one torsion for each rotatable bond; receptor flexibility adds many more. Poses can be generated systematically (grids, FFT), stochastically (Monte Carlo, genetic algorithms), or by incremental construction of the ligand in the site. Scoring uses a simplified, physics-inspired function (next slide), much cheaper than a full force field.
There are two ways to fail, and you should always ask which one happened: a search failure (the correct pose is never generated) or a scoring failure (the correct pose is generated but ranked below wrong ones). In T03 you check this by redocking a ligand with known pose and computing the RMSD of all generated poses.
Remember: docking = search + scoring; failures can come from either.

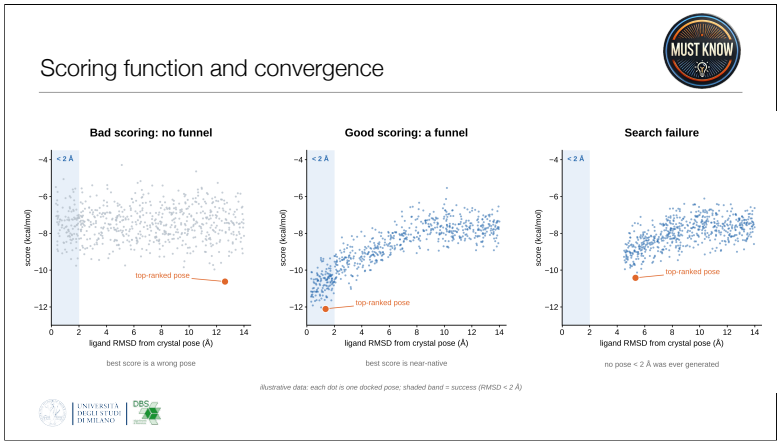



Main idea: a scoring function maps a configuration (all the coordinates) to a single number; Vina uses an empirical function whose form is inspired by physics and whose weights are fitted to experimental data.
Explanation: AutoDock Vina (Trott and Olson, J Comput Chem 2010) sums pairwise terms between receptor and ligand atoms, written as a function of their surface distance: two Gaussian attraction terms (steric/dispersion-like, shape complementarity), a repulsion term (penalises overlaps), a hydrophobic term between hydrophobic atoms (accounts approximately for desolvation), and an H-bond term (also used for metal ions). The sum is then divided by (1 + w·N_rot), where N_rot is the number of rotatable bonds of the ligand: this torsional term is a penalty that approximates the entropy lost when the ligand's flexible bonds are frozen upon binding. (During the search Vina also evaluates intramolecular ligand terms, but they do not enter the final score.) The weights were fitted to reproduce experimental affinities of protein-ligand complexes (PDBbind).
Output: a score in kcal/mol, more negative = better. Accuracy: the correlation with experimental affinities is modest (standard error 2.85 kcal/mol in the original paper, i.e. about a factor 100 in K_D): good for ranking poses, weak for absolute affinities. To be fast, the terms are precomputed on grid maps inside a box around the binding site, which is why you must choose the box in T03.
Remember: score ≈ ΔG_bind, but only approximately; trust it for poses more than for affinities.

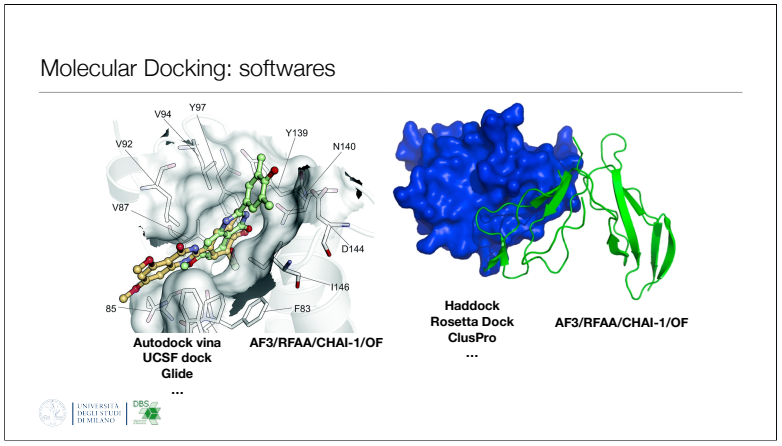



Main idea: docking scores are given in kcal/mol to be read as binding free energies; converting them into K_D shows how approximate they are.
Explanation (Lecture 2): binding is an equilibrium R + L ⇌ RL with K_D = k_off/k_on and

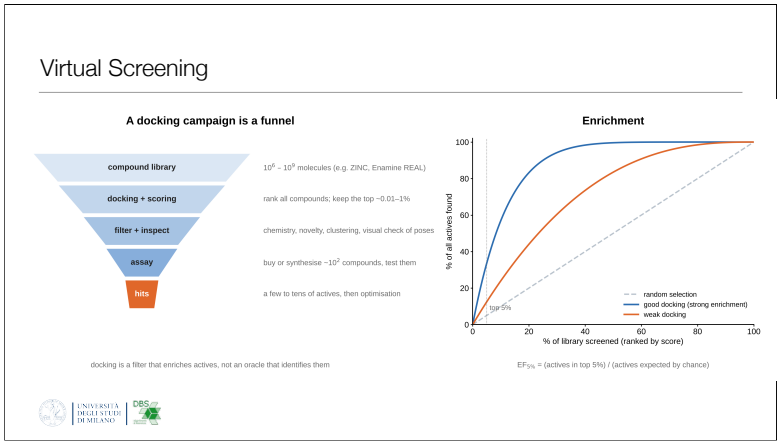
ΔG°_bind = RT ln(K_D/1 M). At 298 K, RT ln 10 ≈ 1.36 kcal/mol: each factor of 10 in K_D is 1.36 kcal/mol. A K_D of 1 mM is about -4.1 kcal/mol, 1 μM about -8.2, 1 nM about -12.3.
Vina's reported standard error on affinities is 2.85 kcal/mol, i.e. about two orders of magnitude in K_D: a score of -9 kcal/mol (K_D ≈ 250 nM) is compatible, within one standard error, with anything from a few nM to tens of μM.
Why: the score misses conformational entropy (only crudely through N_rot), water and desolvation, protein flexibility and protonation states. Scores are useful to rank poses and to filter libraries; affinities require free-energy calculations (alchemical methods, enhanced sampling: Lecture 6) or experiments (ITC, SPR: Lecture 2).
Exercise: which K_D corresponds to -9.5 kcal/mol? (About 100 nM.)
Remember: 1.36 kcal/mol = a factor of 10 in K_D.



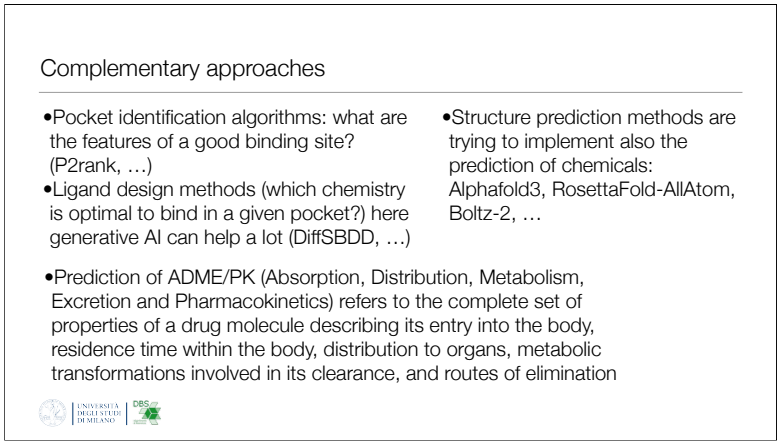

Main idea: a good scoring function must be "funnel-shaped": configurations closer to the true structure must score better.
Explanation: each point is a pose (or a model); x = RMSD from the experimental structure, y = score. Bad: no correlation between score and RMSD, so the lowest score can be any pose. Good: the score decreases as the RMSD goes to zero, with the lowest scores near the native pose (a funnel, as in the energy landscapes of Lecture 2). The same plot is used to evaluate scoring functions in structure prediction (decoy discrimination).
Convergence: independent runs (different random seeds) should return the same best pose; if they do not, the search is not converged and more sampling is needed (in Vina: increase the exhaustiveness). Figure: left, no funnel, the top-ranked pose is 12 Å away (a SCORING failure); centre, a funnel, the top-ranked pose is near-native; right, no pose below 2 Å was ever generated, so no scoring function could succeed (a SEARCH failure). In T03 you can make this plot yourself for the redocked ligand.
Remember: score vs RMSD tells you whether the scoring is reliable; repeated runs tell you whether the search is converged.



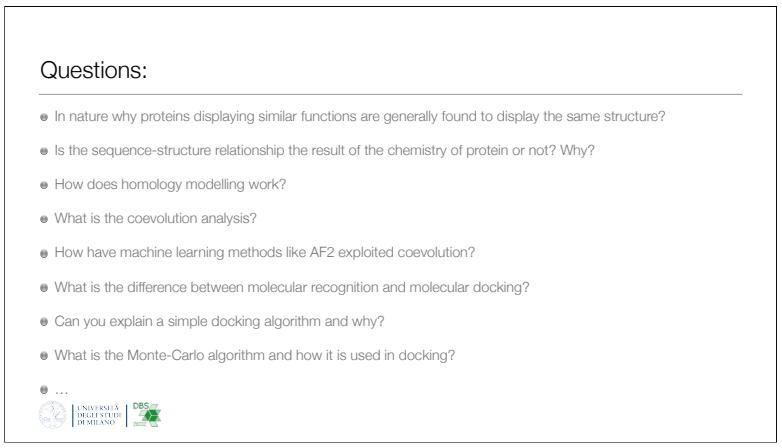

Main idea: an overview of docking programs, organised by problem: small molecules vs macromolecules, classical docking vs co-folding.
Explanation: small-molecule docking: AutoDock Vina (open, used in T03), UCSF DOCK, Glide (commercial). Protein-protein docking: HADDOCK (data-driven: it uses experimental or predicted restraints such as NMR shifts, mutagenesis, crosslinks, coevolution), RosettaDock, ClusPro (FFT rigid-body search). Co-folding methods (see "From single chains to complexes: co-folding"; AlphaFold3, RoseTTAFold All-Atom, Chai-1, OpenFold3, Boltz) predict the whole complex directly from the sequences (and the SMILES of the ligand), without a receptor structure.
Trade-offs: co-folding is often more accurate on PDB-like systems and handles receptor flexibility implicitly, but it can memorise ligands and pockets seen in training and can produce physically implausible poses (check bond lengths, clashes and stereochemistry, e.g. with PoseBusters). Classical docking needs a receptor structure and a binding site, but it is physics-inspired, fast, and suitable for screening millions of compounds.
Remember: choose the tool according to the system and the question; validate whichever tool you use.




Main idea: virtual screening uses docking to rank large libraries of compounds and to discard those likely to be inactive.
Explanation: libraries range from thousands to billions of compounds (e.g. ZINC, Enamine REAL, many not yet synthesised, "make on demand"). Since scores are approximate (see the Vina slide), docking is not used to identify the active molecule with certainty but as a filter: the goal is enrichment, i.e. a much higher fraction of actives among the top-ranked compounds than in the library. The top few hundred compounds are inspected and tested experimentally. Ultra-large campaigns (e.g. Lyu et al., Nature 2019: ~170 million molecules docked against the dopamine D4 receptor and AmpC β-lactamase) showed that hit rates improve with library size.
Alternatives and complements: ligand-based screening (similarity to known actives, pharmacophores) and ML scoring models.
Figure (fig_virtual_screening): the funnel of a campaign, and the enrichment curve: the fraction of all actives found as a function of the fraction of the ranked library screened. Random selection is the diagonal; the enrichment factor EF at 5% is the fraction of actives in the top 5% divided by the 5% expected by chance.
Remember: virtual screening = enrichment, not identification; experiments decide.




Main idea: docking is one step of a broader structure-based drug-discovery pipeline, with complementary computational approaches.
Explanation: (1) pocket identification: where can a ligand bind? P2Rank (machine learning on surface points, used in T03 to define the docking box) or fpocket. (2) Ligand design: which chemistry is optimal for a given pocket? Generative models, e.g. DiffSBDD (diffusion conditioned on the pocket; diffusion models in Lecture 3, design in Lecture 9). (3) Co-folding methods predict protein-ligand complexes directly (AlphaFold3, RoseTTAFold All-Atom, Boltz-2, which also estimates affinities). (4) ADME/PK (absorption, distribution, metabolism, excretion, pharmacokinetics) and toxicity: a good binder is not necessarily a drug; it must reach its target, stay long enough, not be metabolised too fast and not be toxic (e.g. hERG cardiotoxicity, the deephERG example in Lecture 3; Lipinski's rule of five as a first filter). Remember: binding is necessary, not sufficient.




Main idea: questions to check your understanding; they are representative of what you may be asked at the exam.
Further questions:
What is Anfinsen's hypothesis and why does it make structure prediction an optimisation problem? What is Levinthal's paradox?
Why do mutual information and direct information give different contact maps? What is a Potts model and how is it related to the Boltzmann distribution?
What is a distogram, and how can it be transformed into an energy function to obtain a structure?
What are the main components of AlphaFold2 (Evoformer, structure module, recycling) and what does each do?
What do pLDDT, PAE, pTM and ipTM measure? What is the difference between GDT_TS and RMSD?
Give three things that AlphaFold2 cannot tell you.
What is the difference between lock and key, induced fit and conformational selection, and which docking strategy corresponds to each?
Why can docking fail? How do you distinguish a search failure from a scoring failure?
Which K_D corresponds to a binding free energy of -9.5 kcal/mol at room temperature? (About 100 nM.)
What is virtual screening and why is it a filter rather than an identification method?
Next lecture: Molecular Dynamics simulations (Lecture 5): force fields, integration algorithms and analysis, i.e. the physics-based approach that we saw here is limited by sampling.